

Rapport de Projet de Fin d'Etudes

En vue de l'obtention du Diplome National d'Ingenieur / Licence
Appliquee

Conception et mise en oeuvre d'une solution decisionnelle pour l'analyse et l'optimisation du transport urbain de Dakar

Cas du CETUD – Conseil Executif des Transports Urbains de Dakar

Realise par :
Ibtihel Abdellaoui

Encadre par :
[Nom de l'encadrant academique]
[Nom de l'encadrant professionnel –
CETUD]



Remerciements

Au terme de ce travail, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Je remercie en premier lieu mon encadrant académique pour son suivi rigoureux, ses conseils avisés et sa disponibilité tout au long de ce projet.

Mes remerciements vont également à mon encadrant professionnel au sein du CETUD (Conseil Exécutif des Transports Urbains de Dakar) pour la confiance qu'il m'a accordée, pour l'accès aux données du projet et pour son accompagnement durant toute la durée du stage.

Je remercie aussi l'ensemble du corps enseignant qui a contribué à ma formation tout au long de mon parcours universitaire, ainsi que les membres du jury qui me font l'honneur d'évaluer ce travail.

Enfin, je remercie ma famille et mes proches pour leur soutien constant et leurs encouragements.

Résumé

Le présent rapport décrit la conception et la mise en œuvre d’une solution décisionnelle (Business Intelligence) destinée à l’analyse et à l’optimisation du transport urbain de la ville de Dakar, réalisée pour le compte du CETUD (Conseil Exécutif des Transports Urbains de Dakar). Le projet s’appuie sur les données de l’Enquête Ménages-Déplacements (EMD) ainsi que sur des données de comptage de trafic, et couvre l’ensemble de la chaîne décisionnelle : compréhension métier, modélisation multidimensionnelle des données en schéma en constellation (3 faits, 7 dimensions), mise en œuvre d’un processus d’extraction-transformation-chargement (ETL) avec Talend, implémentation d’un entrepôt de données sous PostgreSQL, restitution à travers des tableaux de bord interactifs développés avec Knowage BI, et enrichissement de la solution par des modèles de Machine Learning (segmentation des usagers, système de recommandation, détection d’anomalies de trafic et prédiction des zones à risque d’inaccessibilité). L’ensemble de la solution est exposé à travers une application web (React / FastAPI) destinée à la fois aux usagers du réseau et aux décideurs du CETUD. Ce travail répond à la problématique de la valorisation d’un grand volume de données hétérogènes afin de fournir une vision globale, fiable et exploitable des flux de transport, en vue d’améliorer la prise de décision et l’optimisation du réseau de transport urbain.

Mots-clés : Business Intelligence, Data Warehouse, ETL, Talend, PostgreSQL, Knowage, schéma en constellation, Machine Learning, transport urbain, CETUD, Dakar.

Abstract

This report presents the design and implementation of a Business Intelligence solution for the analysis and optimization of urban transport in Dakar, carried out on behalf of CETUD (Executive Council of Urban Transport of Dakar). The project relies on Household Travel Survey (EMD) data as well as traffic count data, and covers the full decision-support chain: business understanding, multidimensional data modeling using a constellation schema (3 fact tables, 7 dimensions), an ETL process implemented with Talend, a PostgreSQL data warehouse, interactive dashboards built with Knowage BI, and Machine Learning models for user segmentation, recommendation, traffic anomaly detection and accessibility-risk prediction. The whole solution is exposed through a web application (React / FastAPI) for both end users and CETUD decision-makers.

Keywords: Business Intelligence, Data Warehouse, ETL, Talend, PostgreSQL, Knowage, constellation schema, Machine Learning, urban transport, CETUD, Dakar.

Contents

Remerciements	ii
Résumé	iii
Introduction générale	1
1 Étude de l'existant et analyse des besoins	5
1.1 Introduction	5
1.2 Présentation de l'entreprise	5
1.3 Contexte métier	5
1.4 Présentation du domaine du transport urbain	6
1.5 Analyse de l'existant	6
1.5.1 Processus actuel	6
1.5.2 Limites de l'existant	6
1.6 Analyse des besoins	7
1.6.1 Besoins fonctionnels	7
1.6.2 Besoins non fonctionnels	8
1.7 Identification des acteurs	8
1.8 Cas d'utilisation	9
1.9 Conclusion	10
2 Conception de la solution décisionnelle	11
2.1 Introduction	11
2.2 Architecture globale proposée	11
2.3 Choix technologiques	13
2.4 Architecture fonctionnelle	13
2.5 Architecture technique	14
2.6 Architecture logicielle	14
2.7 Modélisation UML	15
2.7.1 Diagramme de cas d'utilisation	15
2.8 Conclusion	15
3 Conception décisionnelle	16

3.1	Introduction	16
3.2	Description des sources de données	16
3.3	Architecture ETL	17
3.4	Processus ETL avec Talend	17
3.4.1	Extraction	17
3.4.2	Nettoyage	17
3.4.3	Transformation	17
3.4.4	Chargement	20
3.5	Conception du Data Warehouse	20
3.5.1	Choix du modèle décisionnel	20
3.5.2	Tables de faits	21
3.5.3	Dimensions	22
3.5.4	Hierarchies	22
3.5.5	Mesures et indicateurs	22
3.5.6	Implémentation physique	23
3.6	Modèle décisionnel final	24
3.7	Conclusion	24
4	Développement de la plateforme décisionnelle	26
4.1	Introduction	26
4.2	Architecture de l'application	26
4.3	Backend FastAPI	26
4.3.1	Organisation du code	26
4.3.2	API REST	27
4.3.3	Gestion des utilisateurs et rôles	27
4.3.4	Authentification	27
4.3.5	Gestion des rôles	28
4.3.6	Export PDF	28
4.3.7	Communication avec PostgreSQL	28
4.3.8	Communication avec les modèles de Machine Learning	28
4.4	Frontend React	28
4.4.1	Organisation des composants	29
4.4.2	Navigation	29
4.4.3	Pages	29
4.4.4	Dashboards et formulaires	30
4.4.5	Intégration de l'API	30
4.5	Intégration de Knowage	30
4.5.1	Datasets	30
4.5.2	Cockpits et widgets	30

4.5.3	KPI restitués	30
4.5.4	Intégration dans React	31
4.6	Sécurité de l'application	31
4.7	Conclusion	31
5	Intégration du Machine Learning	32
5.1	Motivation	32
5.2	Pourquoi un entraînement direct sur les sources	32
5.3	Pipeline de Machine Learning	33
5.3.1	Prétraitement	33
5.3.2	Feature engineering	33
5.3.3	Sélection des variables	33
5.3.4	Entraînement	33
5.3.5	Évaluation	33
5.3.6	Sauvegarde	34
5.4	Présentation des modèles	34
5.4.1	Segmentation des usagers (K-Means)	34
5.4.2	Recommandation de mode de transport (Random Forest)	36
5.4.3	Détection d'anomalies de trafic	37
5.4.4	Prédiction du risque d'inaccessibilité	38
5.5	Intégration dans FastAPI	42
5.6	Exploitation des prédictions	43
5.7	Valeur ajoutée de l'intelligence artificielle	43
5.8	Conclusion	43
6	Résultats et évaluation	44
6.1	Introduction	44
6.2	Présentation des dashboards	44
6.3	Analyse des KPIs	45
6.4	Résultats des modèles ML	45
6.5	Comparaison BI vs ML	46
6.6	Apport global	46
6.7	Limites	46
6.8	Perspectives	47
6.9	Conclusion	47
	Conclusion générale	48
A	Annexes	52
A.1	Scripts SQL	52

A.2	Jobs Talend	52
A.3	Architecture complète	53
A.4	Captures Knowage	53
A.5	Captures React	53
A.6	Documentation de l'API	53
A.7	Documentation des modèles de Machine Learning	54
A.8	Manuel utilisateur	54
A.8.1	Espace citoyen	54
A.8.2	Espace décideurs	55
A.9	Dictionnaire des codes de variables EMD utilisés	55
A.10	Glossaire	55
A.11	Liste des technologies utilisées	56

List of Figures

1	Méthodologie GIMSI appliquée à la construction du système décisionnel . .	3
2	Diagramme de Gantt du projet	3
1.1	Diagramme de cas d'utilisation de la solution	9
2.1	Architecture globale de la solution décisionnelle	12
3.1	Job Talend de chargement de la Staging Area – déplacements	18
3.2	Composant tMap du job de construction de Fact_deplacement	19
3.3	Job Talend de construction de Fact_Comptage	19
3.4	Jobs de construction des dimensions de référence	20
3.5	Schéma en constellation du modèle décisionnel (3 faits, 7 dimensions) . . .	21
3.6	Structure de la table Fact_Comptage (pgAdmin)	23
3.7	Extrait de la table Dim_Geographie (507 zones)	24
5.1	Exploration des données individus avant segmentation	34
5.2	Détermination du nombre optimal de clusters (méthode du coude + silhouette)	35
5.3	Caractérisation des segments par variable (heatmap)	35
5.4	Matrice de confusion du modèle de recommandation de mode	36
5.5	Tableau de bord de synthèse des anomalies de trafic détectées	38
5.6	Cartographie des zones classées par niveau de risque d'inaccessibilité	40
5.7	Importance des variables du modèle de prédiction d'inaccessibilité	41

5.8	Analyse d'explicabilité SHAP du modèle de prédiction d'inaccessibilité . . .	42
6.1	Identité visuelle du cockpit décisionnel TransportDakar	44
A.1	Architecture technique complète de la solution	53

List of Tables

1.1	Synthèse des acteurs et de leurs objectifs	9
2.1	Choix technologiques de la solution	13
3.1	Dictionnaire des dimensions du modèle décisionnel	22
4.1	Principaux endpoints de l'API FastAPI	27
4.2	Pages principales du frontend	29
5.1	Segments d'utilisateurs identifiés par K-Means	35
5.2	Algorithmes de détection d'anomalies et pondération du consensus	37
5.3	Performances du modèle de prédiction d'inaccessibilité (Gradient Boosting)	39
6.1	Synthèse des indicateurs clés produits par la solution	45
6.2	Synthèse des performances des modèles de Machine Learning	45
6.3	Comparaison des apports de la BI classique et du Machine Learning	46
A.1	Jobs Talend développés	52
A.2	Liste complète des endpoints REST exposés par l'API	54
A.3	Principaux codes de variables EMD exploités	55
A.4	Stack technique du projet	56

Introduction générale

Contexte

La ville de Dakar connaît, comme la plupart des grandes agglomérations africaines, une croissance urbaine rapide accompagnée d'une demande de mobilité de plus en plus forte. Le réseau de transport urbain, structuré autour de multiples modes (bus, cars rapides, Ndiaga Ndiaye, TATA, taxis, transport informel), génère un volume important de données issues des enquêtes ménages-déplacements et des dispositifs de comptage de trafic. Ces données, bien que riches, restent aujourd'hui largement sous-exploitées par manque d'outils permettant de les centraliser, de les croiser et de les restituer sous une forme directement utilisable par les décideurs.

C'est dans ce contexte que le CETUD (Conseil Exécutif des Transports Urbains de Dakar), autorité organisatrice des transports urbains de la région de Dakar, a souhaité disposer d'une solution décisionnelle complète permettant de transformer ces données brutes en informations stratégiques. L'enjeu n'est pas uniquement technique : il s'agit de fournir au directeur de planification et au chef d'exploitation du CETUD une vision fiable, actualisée et exploitable des flux de transport, afin d'orienter les décisions d'investissement, d'adaptation de l'offre et d'amélioration de l'accessibilité.

Problématique

La problématique générale ayant motivé ce projet peut être formulée ainsi :

Comment exploiter efficacement un grand volume de données hétérogènes issues du système de transport urbain afin de fournir une vision globale, pertinente et exploitable des flux de transport, et ainsi améliorer la prise de décision et l'optimisation du réseau de transport urbain ?

Cette problématique se décline en plusieurs questions opérationnelles :

- Comment garantir la qualité, la fiabilité et la cohérence de données collectées depuis des sources multiples et hétérogènes (enquêtes terrain, comptages automatiques) ?
- Comment structurer ces données pour qu'elles soient directement exploitables dans des indicateurs décisionnels ?
- Comment restituer ces indicateurs de façon claire et actionnable aux différents profils de décideurs du CETUD ?

- Comment aller au-delà du simple reporting descriptif, en exploitant les données pour anticiper (segmentation des usagers, détection d'anomalies, prédiction des zones à risque) ?

Objectifs du projet

Pour répondre à cette problématique, le projet poursuit les objectifs suivants :

1. **Compréhension métier et données** : analyser les sources de données disponibles (EMD, comptages de trafic) et identifier les besoins des décideurs du CETUD.
2. **Modélisation décisionnelle** : concevoir un modèle multidimensionnel capable de répondre aux besoins analytiques identifiés.
3. **Mise en œuvre de l'ETL** : industrialiser la collecte, le nettoyage et le chargement des données dans l'entrepôt de données à l'aide de Talend.
4. **Implémentation de l'entrepôt de données** : déployer un entrepôt de données sous PostgreSQL conforme au modèle décisionnel conçu.
5. **Visualisation et restitution** : développer des tableaux de bord interactifs avec Knowage BI permettant le suivi des indicateurs clés.
6. **Enrichissement par le Machine Learning** : développer des modèles prédictifs et de segmentation pour aller au-delà du reporting descriptif.
7. **Intégration applicative** : exposer l'ensemble de la solution à travers une application web destinée aux usagers et aux décideurs.

Méthodologie

La conduite du projet s'est appuyée sur la méthodologie GIMSI (Generalized Indicator for Management of Strategic Information), une démarche structurée en quatre étapes destinée à la construction de systèmes de tableaux de bord : identification, conception, mise en œuvre et amélioration permanente. Cette méthodologie a guidé en particulier la démarche de définition des indicateurs en partant de l'identification des décideurs et de leurs objectifs métier, plutôt que de partir directement des données disponibles.

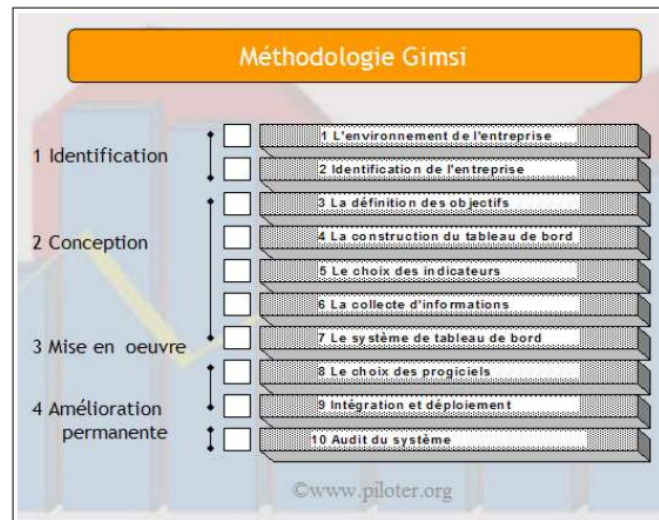


Figure 1: Méthodologie GIMSI appliquée à la construction du système décisionnel

Sur le plan du développement logiciel, une approche itérative inspirée de Scrum a été adoptée : le projet a été découpé en cycles courts correspondant chacun à une brique de la chaîne décisionnelle (analyse, modélisation, ETL, entrepôt, restitution, Machine Learning, application web), avec des points d'avancement réguliers avec l'encadrant professionnel du CETUD. Le projet s'est déroulé sur une période de six mois, structurée selon le planning suivant :

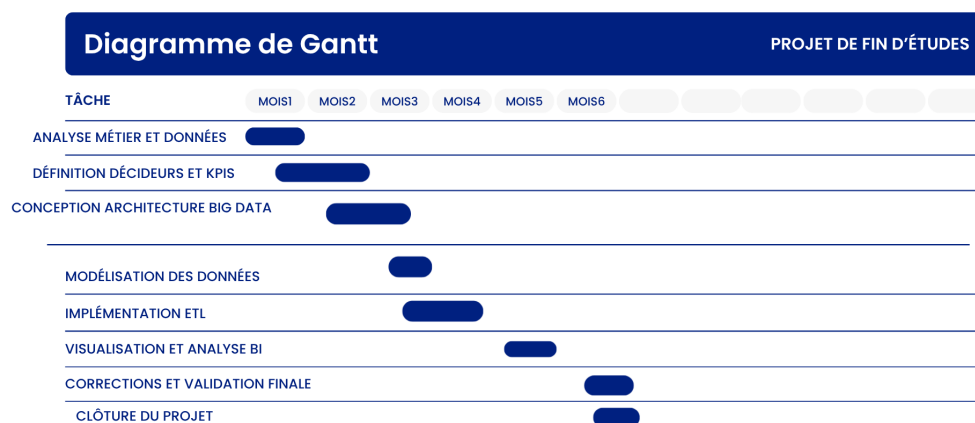


Figure 2: Diagramme de Gantt du projet

Organisation du mémoire

Ce rapport est organisé en six chapitres. Le premier chapitre présente l'étude de l'existant et l'analyse des besoins : l'organisme d'accueil, le domaine du transport urbain, les limites

du processus actuel et le cahier des charges fonctionnel et non fonctionnel. Le deuxième chapitre décrit la conception de la solution décisionnelle : l'architecture globale proposée, les choix technologiques et la modélisation UML retenue. Le troisième chapitre détaille la conception décisionnelle proprement dite : sources de données, architecture ETL et conception de l'entrepôt de données. Le quatrième chapitre couvre le développement de la plateforme décisionnelle : backend FastAPI, frontend React, intégration de Knowage et sécurité. Le cinquième chapitre est consacré à l'intégration du Machine Learning, des choix méthodologiques jusqu'à l'exploitation des prédictions dans l'application. Le sixième chapitre présente les résultats obtenus et leur évaluation. Le rapport se termine par une conclusion générale, une bibliographie et un ensemble d'annexes techniques.

Chapter 1

Étude de l'existant et analyse des besoins

1.1 Introduction

Conformément à la première étape de la méthodologie GIMSI (identification), ce chapitre pose le cadre métier du projet : présentation de l'organisme d'accueil, du domaine du transport urbain à Dakar, analyse du processus décisionnel actuel et de ses limites, puis spécification des besoins fonctionnels et non fonctionnels qui ont guidé la conception de la solution.

1.2 Présentation de l'entreprise

Le CETUD (Conseil Exécutif des Transports Urbains de Dakar) est l'autorité organisatrice des transports urbains pour la région de Dakar. Créé pour mettre en œuvre la politique de l'État en matière de transport urbain, il a pour mission de planifier, organiser et réguler l'offre de transport public sur le territoire de l'agglomération dakaroise, en coordonnant les différents opérateurs (bus Dakar Dem Dikk, cars rapides, Ndiaga Ndiaye, TATA, transport informel) et en veillant à l'amélioration continue de l'accessibilité et de la qualité de service offerte aux usagers.

Dans le cadre de ses missions de planification, le CETUD réalise et exploite des enquêtes de grande ampleur, notamment l'Enquête Ménages-Déplacements (EMD), ainsi que des campagnes de comptage de trafic sur les principaux axes et points névralgiques du réseau. Ces données constituent un patrimoine informationnel considérable, mais dont l'exploitation reste aujourd'hui largement manuelle et ponctuelle.

1.3 Contexte métier

La mobilité urbaine à Dakar se caractérise par une forte hétérogénéité de l'offre (transport formel et informel coexistants), une demande en croissance continue liée à l'étalement urbain, et des contraintes d'accessibilité marquées dans certaines zones périphériques. Le CETUD doit, dans ce contexte, arbitrer en permanence entre plusieurs préoccupations : l'adaptation de l'offre de transport en commun à la demande réelle, la priorisation des

investissements d'infrastructure, et le suivi de la qualité de service perçue par les usagers. Ces arbitrages reposent aujourd'hui largement sur l'expertise des agents du CETUD et sur des extractions ponctuelles de données, plutôt que sur un système d'information décisionnel structuré.

1.4 Présentation du domaine du transport urbain

Le réseau de transport urbain de Dakar repose sur plusieurs catégories de données qu'il convient de bien distinguer :

- l'**Enquête Ménages-Déplacements (EMD)**, une enquête déclarative menée auprès d'un échantillon de ménages, recensant les caractéristiques socio-démographiques des individus et le détail de leurs déplacements quotidiens (motif, mode, durée, coût, origine, destination) ;
- les **comptages de trafic**, des relevés automatiques ou manuels de flux de véhicules effectués sur des sites de comptage répartis sur le réseau, à une granularité horaire ;
- les **indicateurs d'accessibilité**, dérivés des deux sources précédentes, qui qualifient la qualité de la desserte en transport en commun par zone géographique.

Ce triptyque de données (déplacements déclarés, trafic observé, accessibilité dérivée) constitue le socle informationnel sur lequel s'appuie l'ensemble de la solution décisionnelle développée dans ce projet.

1.5 Analyse de l'existant

1.5.1 Processus actuel

Avant ce projet, l'exploitation des données EMD et de comptage de trafic au CETUD suivait un processus essentiellement manuel : les fichiers bruts (extraits Excel/CSV des enquêtes) étaient consultés au cas par cas, généralement à l'occasion d'une étude ponctuelle ou d'une demande spécifique d'un service. Les calculs d'indicateurs (volumes, taux, scores) étaient réalisés directement dans des feuilles de calcul, sans modèle de données partagé ni historique structuré. Il n'existait pas de tableau de bord consolidé permettant un suivi régulier des indicateurs clés, ni d'outil permettant de croiser facilement les données de déplacement avec les données de trafic ou les indicateurs d'accessibilité.

1.5.2 Limites de l'existant

Cette situation présentait plusieurs limites structurelles :

- **Absence de modèle de données partagé** : chaque analyse repartait des fichiers bruts, sans réutilisation d'un référentiel commun (dimensions, mesures), ce qui générerait des incohérences entre analyses successives ;
- **Manque de centralisation** : les données EMD, les données de comptage et les indicateurs d'accessibilité étaient traités séparément, rendant difficile toute analyse croisée ;
- **Restitution non standardisée** : les résultats étaient produits sous forme de documents ponctuels (tableaux, graphiques isolés), sans interface de consultation interactive ni mise à jour automatisée ;
- **Aucune dimension prédictive** : l'analyse se limitait à du reporting descriptif (comptages, moyennes), sans capacité de segmentation des usagers, de détection d'anomalies ou de prédiction des zones à risque ;
- **Dépendance à l'expertise individuelle** : la connaissance des règles de calcul et des spécificités des données reposait sur quelques personnes, sans documentation centralisée.

Ces limites justifient la mise en place d'une chaîne décisionnelle structurée, allant de l'ETL jusqu'à la restitution interactive, enrichie par des modèles de Machine Learning permettant de dépasser le seul reporting descriptif.

1.6 Analyse des besoins

1.6.1 Besoins fonctionnels

L'analyse menée avec les décideurs du CETUD a permis d'identifier les besoins fonctionnels suivants :

- centraliser les données issues de l'EMD et des comptages de trafic dans un entrepôt de données unique et structuré ;
- produire automatiquement les indicateurs clés de performance (KPI) nécessaires au suivi de la mobilité urbaine (volumes de déplacement, heures de pointe, scores d'accessibilité, etc.) ;
- restituer ces indicateurs sous forme de tableaux de bord interactifs, navigables par thématique (trafic, déplacements, démographie, accessibilité) ;
- permettre la consultation de prédictions et d'analyses avancées (segmentation des usagers, détection d'anomalies de trafic, prédiction du risque d'inaccessibilité, recommandation de mode de transport) ;

- offrir un espace public permettant au citoyen d'obtenir une recommandation personnalisée de mode de transport ;
- distinguer les droits d'accès selon le profil de l'utilisateur (citoyen, chef d'exploitation, directeur de planification).

1.6.2 Besoins non fonctionnels

Au-delà des fonctionnalités, plusieurs exigences non fonctionnelles ont structuré les choix de conception :

- **Performance** : les temps de réponse des tableaux de bord et de l'API de recommandation doivent rester compatibles avec un usage interactif (chargement des modèles de Machine Learning en mémoire au démarrage du serveur plutôt qu'à chaque requête) ;
- **Sécurité** : l'accès à l'espace décideurs doit être protégé par authentification, avec un contrôle d'accès basé sur les rôles (RBAC) ;
- **Ergonomie** : les interfaces doivent être accessibles à des utilisateurs non techniques, avec une navigation claire et un mode sombre/clair ;
- **Maintenabilité** : l'architecture doit séparer clairement les couches (ETL, entrepôt, restitution BI, application web) afin de faciliter l'évolution indépendante de chaque composant ;
- **Traçabilité** : chaque indicateur restitué doit pouvoir être retracé jusqu'à sa source dans l'entrepôt de données, condition de la confiance des décideurs dans le système.

1.7 Identification des acteurs

Trois acteurs principaux interagissent avec la solution :

- **Citoyen / usager** : tout usager du réseau de transport, sans authentification requise, qui consulte l'application pour obtenir une recommandation de mode de transport adaptée à son profil de mobilité ;
- **Chef d'Exploitation** : décideur opérationnel du CETUD, authentifié, qui consulte les tableaux de bord de suivi (trafic, déplacements, zones à risque, anomalies, segmentation des usagers) afin d'ajuster l'offre de transport au quotidien ;
- **Directeur Planification** : décideur stratégique du CETUD, authentifié, qui dispose des mêmes accès que le chef d'exploitation, complétés par l'accès aux pages

de transparence et d'audit des modèles de Machine Learning (métriques de performance, importance des variables), dans un objectif de gouvernance des décisions automatisées.

Table 1.1: Synthèse des acteurs et de leurs objectifs

Acteur	Authentification	Objectif principal
Citoyen / usager	Non	Obtenir une recommandation de mode de transport
Chef d'Exploitation	Oui (RBAC)	Ajuster l'offre au quotidien à partir des tableaux de bord
Directeur Planification	Oui (RBAC)	Piloter la stratégie et auditer les modèles de ML

1.8 Cas d'utilisation

Le diagramme de cas d'utilisation de la figure 1.1 synthétise les fonctionnalités offertes à chacun des trois acteurs identifiés. Les fonctionnalités communes aux deux décideurs (consultation des tableaux de bord trafic, déplacements, démographie, accessibilité, zones à risque, anomalies et segmentation, ainsi que l'export PDF) reposent toutes sur l'authentification (UC4). Seul le directeur de planification accède en complément à l'audit du modèle de Machine Learning (UC15), conformément à son rôle de gouvernance.

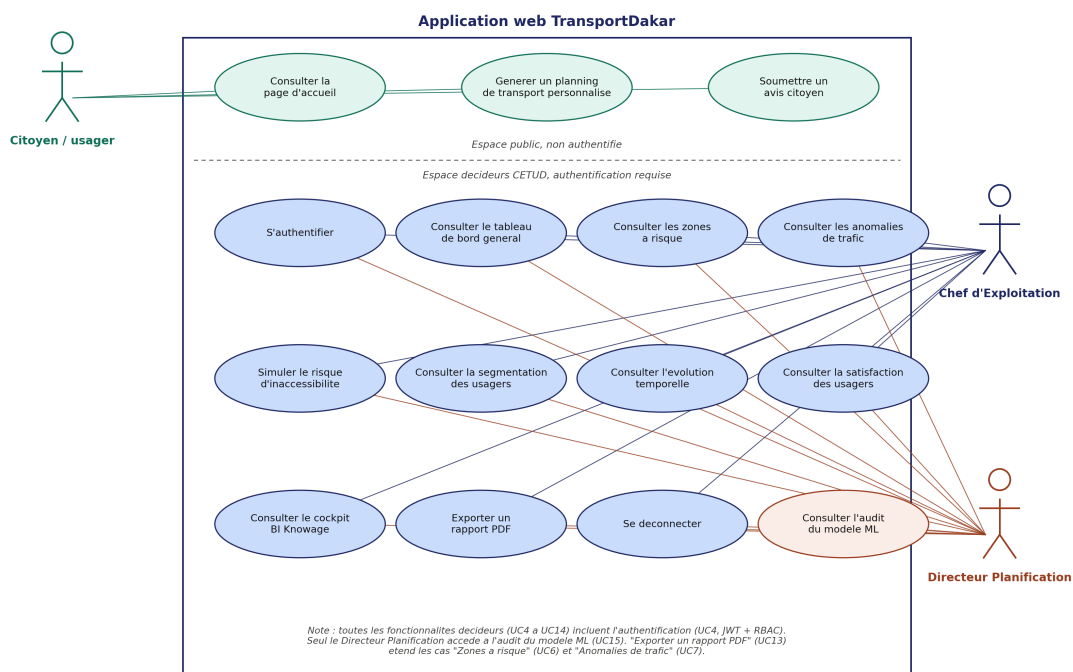


Figure 1.1: Diagramme de cas d'utilisation de la solution

1.9 Conclusion

Ce chapitre a permis de poser le cadre métier du projet : présentation du CETUD et du domaine du transport urbain, analyse critique du processus décisionnel existant, et spécification précise des besoins fonctionnels et non fonctionnels ainsi que des acteurs de la solution. Ces éléments constituent le cahier des charges à partir duquel la conception de la solution décisionnelle, objet du chapitre suivant, a été élaborée.

Chapter 2

Conception de la solution décisionnelle

2.1 Introduction

Conformément à la deuxième étape de la méthodologie GIMSI (conception), ce chapitre présente la conception globale de la solution : l'architecture générale retenue, les choix technologiques opérés à chaque niveau de la chaîne, les architectures fonctionnelle, technique et logicielle, ainsi que la modélisation UML de la solution.

2.2 Architecture globale proposée

La solution repose sur une architecture en chaîne décisionnelle complète, allant des sources de données brutes jusqu'à leur restitution, doublée d'une couche applicative web exposant les résultats à deux publics distincts (citoyens et décideurs). Cette architecture s'articule autour de quatre grands blocs :

1. un bloc **Sources & ETL** : fichiers sources EMD et comptages de trafic, nettoyés en Python, puis intégrés dans l'entrepôt de données via des jobs Talend ;
2. un bloc **Entrepôt de données** : base PostgreSQL structurée selon un modèle décisionnel multidimensionnel (chapitre 3) ;
3. un bloc **Restitution et intelligence** : cockpit Knowage BI pour le reporting et la visualisation, complété par des modèles de Machine Learning entraînés directement sur les sources nettoyées ;
4. un bloc **Application web** : backend FastAPI exposant une API REST, consommée par un frontend React, qui intègre à la fois les pages applicatives et le cockpit Knowage.

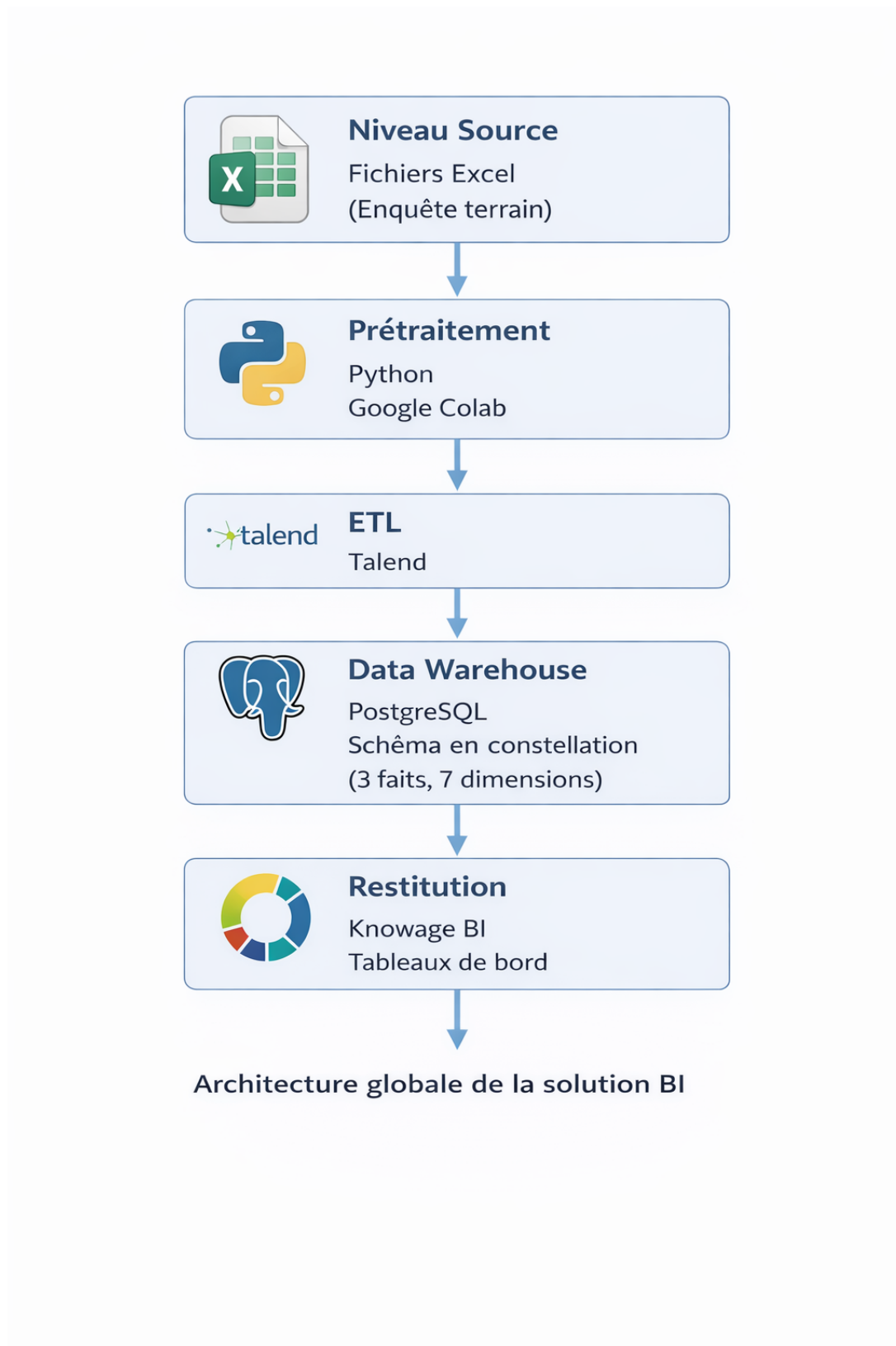


Figure 2.1: Architecture globale de la solution décisionnelle

2.3 Choix technologiques

Le tableau 2.1 récapitule les choix technologiques retenus pour chaque maillon de la chaîne, ainsi que leur justification.

Table 2.1: Choix technologiques de la solution

Couche	Technologie	Justification
Préparation des données	Python / Pandas (Colab)	Écosystème mature pour le nettoyage et l'exploration de données tabulaires volumineuses
ETL	Talend Open Studio	Outil graphique gratuit, composants prêts à l'emploi (tMap, tDBOutput) adaptés à un projet académique
Entrepôt de données	PostgreSQL	SGBDR open source, robuste, bien documenté, compatible avec Knowage et Talend
Restitution BI	Knowage BI	Suite BI open source complète (datasets, cockpits, widgets) sans coût de licence
Machine Learning	Python / scikit-learn	Bibliothèque de référence pour les algorithmes de classification, clustering et détection d'anomalies, avec un bon compromis simplicité/performance
Backend applicatif	FastAPI	Framework Python moderne, performant, génération automatique de documentation interactive (Swagger), validation native via Pydantic
Frontend applicatif	React.js	Bibliothèque la plus répandue pour la construction d'interfaces utilisateur réactives et composables

Les bibliothèques de Machine Learning mobilisées incluent notamment **scikit-learn** (K-Means, Random Forest, Isolation Forest, Local Outlier Factor, DBSCAN, Gradient Boosting), **imbalanced-learn** (SMOTE) et **joblib** pour la sérialisation des modèles entraînés.

2.4 Architecture fonctionnelle

D'un point de vue fonctionnel, la solution distingue trois grandes fonctions : la **fonction décisionnelle classique** (ETL, entrepôt, cockpit Knowage), qui répond aux besoins de reporting et de suivi des KPI ; la **fonction prédictive** (modèles de Machine Learning), qui enrichit l'analyse par des capacités de segmentation, de détection d'anomalies et de

prédiction ; et la **fonction applicative**, qui expose ces deux fonctions à travers des interfaces adaptées à chaque type d'utilisateur (citoyen, chef d'exploitation, directeur de planification). Ces trois fonctions communiquent entre elles : le cockpit Knowage lit l'entrepôt de données, tandis que les modèles de Machine Learning lisent directement les sources nettoyées (ce choix est justifié au chapitre 5), et l'application web orchestre l'accès à l'ensemble via son backend.

2.5 Architecture technique

Sur le plan technique, la solution est déployée selon une architecture en couches :

- une couche **stockage** (PostgreSQL, schéma DW) ;
- une couche **intégration** (Talend, exécuté en mode batch pour alimenter l'entrepôt) ;
- une couche **intelligence** (scripts Python entraînant et sérialisant les modèles de Machine Learning au format `.pkl`) ;
- une couche **services** (serveur FastAPI exposant l'API REST et chargeant les modèles en mémoire au démarrage) ;
- une couche **présentation** (application React consommant l'API REST et intégrant le cockpit Knowage par *embedding*) ;
- une couche **restitution BI** (serveur Knowage, consulté directement ou via le frontend React).

Cette séparation en couches permet à chaque composant d'évoluer indépendamment et facilite la maintenance de la solution dans son ensemble.

2.6 Architecture logicielle

L'architecture logicielle de l'application web suit un modèle client-serveur classique à trois couches : une couche de **présentation** (composants React organisés en pages publiques et pages décideurs protégées), une couche **métier/API** (endpoints FastAPI organisés par domaine fonctionnel : recommandation, zones à risque, anomalies, segmentation, authentification), et une couche **données** (modèles de Machine Learning sérialisés, accès à l'entrepôt PostgreSQL). La communication entre le frontend et le backend s'effectue exclusivement via des requêtes HTTP au format JSON, ce qui découple complètement les deux couches et permettrait, à terme, de remplacer l'une ou l'autre sans impact sur le reste du système.

2.7 Modélisation UML

2.7.1 Diagramme de cas d'utilisation

La modélisation UML de la solution s'appuie sur un diagramme de cas d'utilisation, qui synthétise les interactions entre les trois acteurs identifiés au chapitre précédent (citoyen, chef d'exploitation, directeur de planification) et les fonctionnalités du système. Ce diagramme, déjà présenté en figure 1.1, met en évidence la dépendance de toutes les fonctionnalités décideurs envers l'authentification (UC4), ainsi que la fonctionnalité d'audit des modèles de Machine Learning (UC15), réservée au directeur de planification.

Ce choix de se concentrer sur le diagramme de cas d'utilisation, plutôt que de multiplier les diagrammes UML (séquence, classes, composants, déploiement), répond à un objectif de clarté : le diagramme de cas d'utilisation est le plus pertinent pour exprimer le périmètre fonctionnel réel de l'application et les droits d'accès associés à chaque acteur, et constitue la base de référence pour le développement présenté au chapitre 4.

2.8 Conclusion

Ce chapitre a présenté la conception globale de la solution décisionnelle : son architecture, les technologies retenues à chaque niveau, et sa modélisation par cas d'utilisation. Les chapitres suivants détaillent successivement la mise en œuvre de chacun des blocs de cette architecture : la conception décisionnelle (chapitre 3), le développement de la plateforme (chapitre 4) et l'intégration du Machine Learning (chapitre 5).

Chapter 3

Conception décisionnelle

3.1 Introduction

Ce chapitre détaille la conception décisionnelle de la solution : les sources de données exploitées, l'architecture du processus ETL, sa mise en œuvre avec Talend, et la conception de l'entrepôt de données qui constitue le socle de l'ensemble de la chaîne de restitution.

3.2 Description des sources de données

La principale source de données exploitée est l'**Enquête Ménages-Déplacements (EMD)** de Dakar, complétée par des données de **comptage de trafic** issues de capteurs et de relevés sur les principaux sites de comptage du réseau. L'EMD est structurée en plusieurs blocs de variables, identifiées par des codes (I2, I4, I9, M21, M26, M55, M66 à M91, etc.), couvrant :

- les caractéristiques géographiques du domicile (quartier, strate, district de recensement) ;
- les caractéristiques du ménage (taille, type de logement, équipement) ;
- les déplacements individuels (motif, mode, durée, coût) ;
- l'accessibilité aux transports en commun et aux services (école, daara) ;
- les problèmes et manquements perçus dans le quartier (manque de transport, inondations, enclavement, insécurité).

Ces données sources ont fait l'objet d'une priorisation en trois phases afin de structurer le développement de l'ETL de façon incrémentale :

1. **Phase 1 – MVP** : colonnes géographiques (I2, I4, I9), accessibilité de base (M66 à M70), problèmes majeurs (M73, M87) et taille du ménage (M21) ;
2. **Phase 2 – Extension services** : variables d'accès aux écoles et daaras (M90, M91) ;

3. **Phase 3 – Détails avancés** : variables de résilience en cas de pluie (M69), modes de transport vers les services, et autres problèmes de quartier (bruit, insécurité, accidents).

3.3 Architecture ETL

La chaîne ETL est organisée en deux niveaux successifs :

1. une **zone de préparation (Staging Area, SA)** : jobs chargeant les données brutes, préalablement nettoyées sous Python/Colab, dans des tables intermédiaires, sans transformation lourde ;
2. une **zone de chargement final** : jobs lisant les tables de la Staging Area, appliquant les transformations, jointures et calculs nécessaires (via le composant `tMap`), puis chargeant les dimensions et les faits dans l'entrepôt de données (DW).

Cette architecture à deux niveaux a été retenue afin de séparer clairement le chargement brut (rapide, peu de logique) de la transformation (jointures de référentiel, résolution de clés étrangères, calculs de mesures), ce qui facilite le débogage et la traçabilité de chaque étape.

3.4 Processus ETL avec Talend

3.4.1 Extraction

L'extraction est réalisée par les composants `tFileInputExcel` et `tFileInputDelimited`, qui lisent les fichiers sources (extraits EMD et comptages de trafic), déjà nettoyés en amont via des scripts Python (traitement des valeurs manquantes, normalisation des types, suppression des doublons).

3.4.2 Nettoyage

Au sein de Talend, un nettoyage complémentaire est appliqué via le composant `tFilterColumns`, qui sélectionne uniquement les colonnes utiles conformément à la priorisation définie en section 3 ci-dessus, réduisant ainsi la complexité des jobs et le volume de données transporté.

3.4.3 Transformation

La transformation constitue le cœur du processus ETL, portée par le composant `tMap`. Trois jobs principaux assurent d'abord l'alimentation de la zone de préparation : `SA_individu`, `SA_menage` et `SA_deplacement`, correspondant aux trois grandes entités sources de l'EMD.

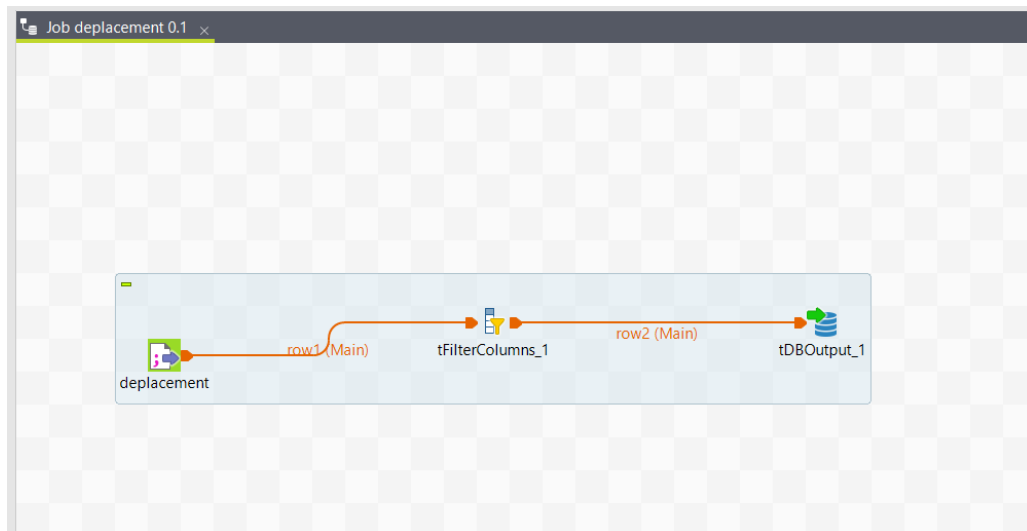


Figure 3.1: Job Talend de chargement de la Staging Area – déplacements

Le job `Job_deplacement` illustre ensuite la logique de transformation appliquée pour construire la table de faits `Fact_deplacement`. Le composant `tMap` central réalise les opérations suivantes :

- récupération du flux principal contenant les attributs du déplacement (numéro de ménage, numéro d'individu, lieux et heures de départ/arrivée, durée, motif, fréquence de transport en commun, mode, coût total) ;
- trois jointures (*lookup*) sur les dimensions `Dim_Motif` (par motif), `Dim_Individu` (par clé composite `num_individu + num_menage`) et `Dim_Mode` (par mode) afin de résoudre les clés étrangères ;
- construction du flux de sortie combinant les clés étrangères résolues et les mesures (durée du trajet, lieux, coût total, heures de départ/arrivée, fréquence de transport en commun).

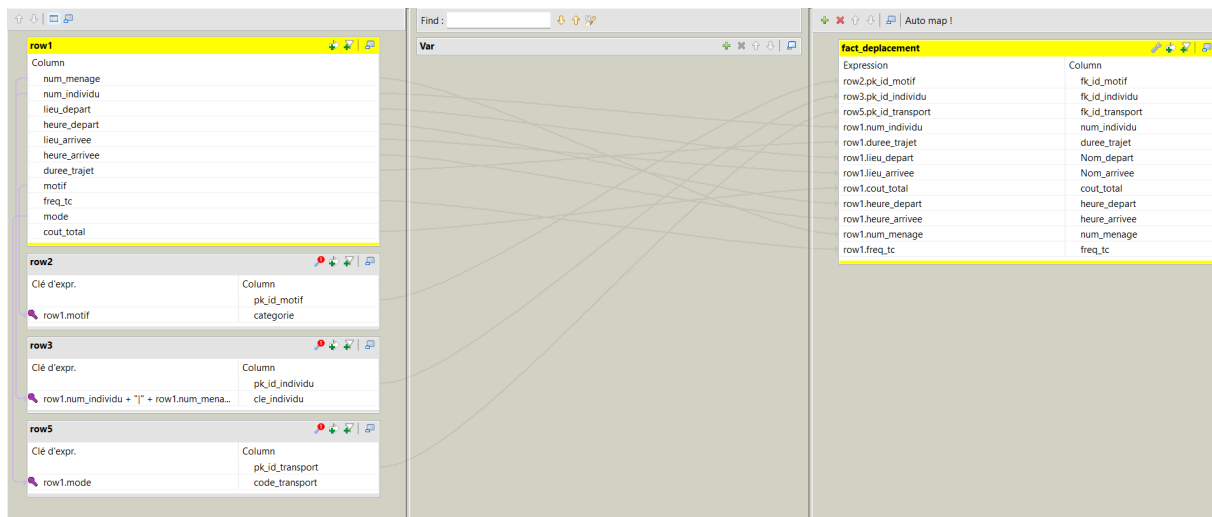


Figure 3.2: Composant tMap du job de construction de Fact_deplacement

Le job de construction de **Fact_Comptage** suit une logique similaire : une entrée principale portant les mesures de comptage est enrichie par deux jointures de référence via un composant **tMap**, avant chargement final. Les jobs **fact_accessibilite** et **fact_indivmen** suivent le même schéma général, en exploitant respectivement les variables d'accessibilité (durée de marche, modes disponibles, exposition aux inondations) et les variables croisées individu/ménage (taille du ménage, équipement, motorisation).

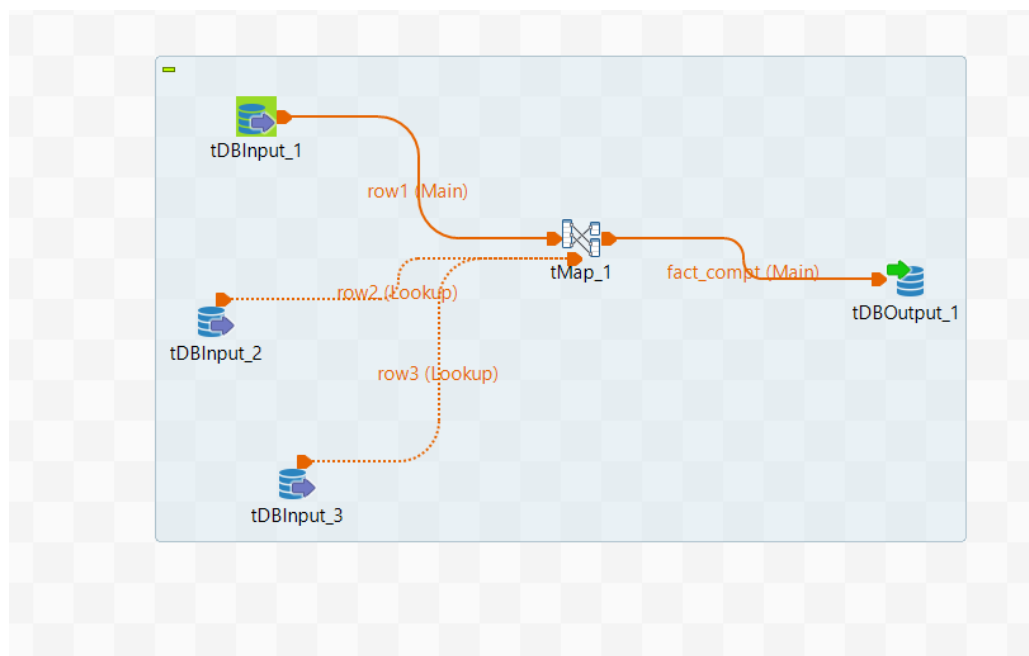


Figure 3.3: Job Talend de construction de Fact_Comptage

3.4.4 Chargement

Le chargement final dans l'entrepôt de données est réalisé par le composant `tDBOutput`, qui écrit successivement les dimensions de référence (`Dim_Site`, `Dim_Geographie`, `Dim_Individu`, `Dim_Menage`) puis les tables de faits, afin de garantir l'intégrité référentielle au moment du chargement.

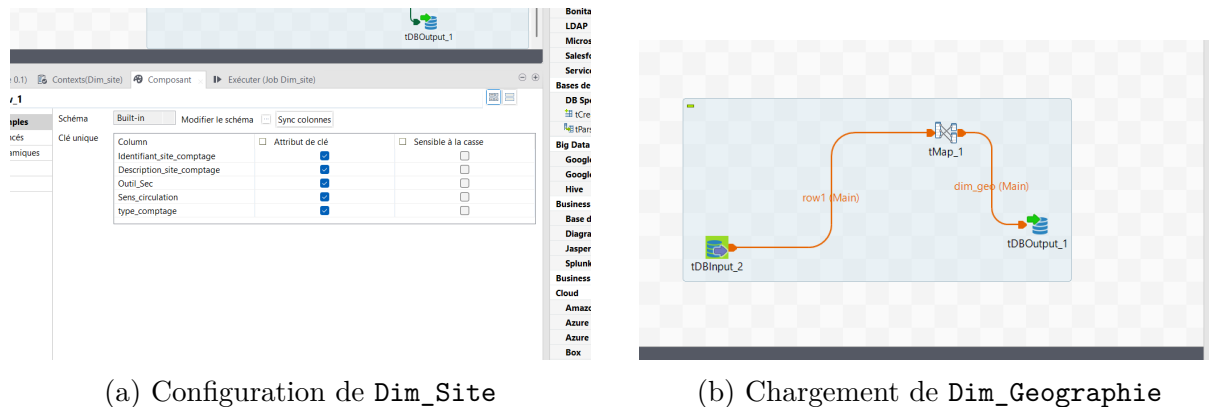


Figure 3.4: Jobs de construction des dimensions de référence

3.5 Conception du Data Warehouse

3.5.1 Choix du modèle décisionnel

La modélisation multidimensionnelle de l'entrepôt s'appuie sur un **schéma en constellation**, c'est-à-dire un modèle comportant plusieurs tables de faits partageant un ensemble commun de dimensions. Ce choix, plutôt qu'un schéma en étoile classique limité à une seule table de faits, est justifié par la nature même des données du projet : les données de comptage de trafic (issues de capteurs, granularité site/heure) et les données d'enquête ménages-déplacements (granularité individu/déplacement) correspondent à des processus métier distincts, avec des cycles de mise à jour différents, mais partagent des axes d'analyse communs (temps, géographie, site). Un schéma en étoile unique aurait nécessité de forcer ces deux processus métier dans une seule table de faits, au prix d'une perte de granularité ou de la création de mesures non applicables selon le contexte ; le schéma en constellation permet au contraire de conserver une table de faits dédiée à chaque processus, tout en mutualisant les dimensions partagées.

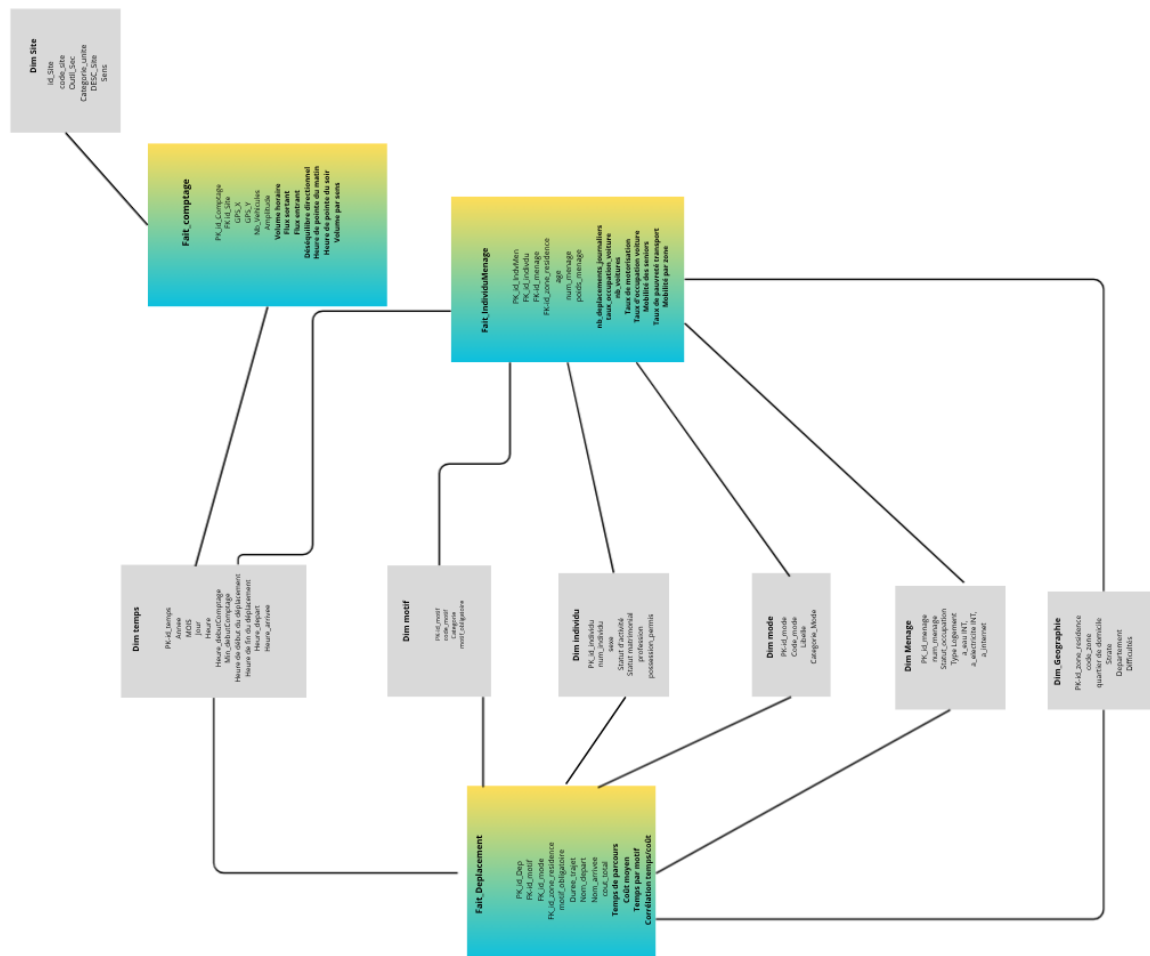


Figure 3.5: Schéma en constellation du modèle décisionnel (3 faits, 7 dimensions)

3.5.2 Tables de faits

Le modèle retenu comporte trois tables de faits.

Fait_Comptage. Enregistre les mesures issues des dispositifs de comptage automatique de trafic, à la granularité du site et de l'heure (clés étrangères : FK_id_Site, FK_id_temps ; mesures : nombre de véhicules, volume horaire matin/soir, flux entrant, flux sortant, désagrégation directionnelle).

Fait_Déplacement. Décrit chaque déplacement individuel déclaré, à la granularité du déplacement (clés étrangères : FK_id_motif, FK_id_mode, FK_id_zone_residence ; mesures : durée du trajet, heures de départ/arrivée, coût total, temps de parcours par motif).

Fait_IndividuMenage. Croise les informations individuelles et celles du ménage, à la granularité de l'individu enquêté (clés étrangères : FK_id individu, FK_id menage,

FK_id_zone_residence ; mesures : nombre de déplacements journaliers, taux d'équipement en voiture, taux de motorisation, mobilité par zone).

3.5.3 Dimensions

Le modèle comporte sept dimensions, dont le dictionnaire est présenté au tableau 3.1.

Table 3.1: Dictionnaire des dimensions du modèle décisionnel

Dimension	Attributs principaux	Granularité
Dim_Site	id_Site, code_site, Ouvl_Soc, catégorie_unité, DESC_Site, Sens	Site de comptage
Dim_Temps	id_temps, année, mois, jour, heure, heure début/fin déplacement	Heure / jour
Dim_Motif	id_motif, code_motif, motif_origine	Motif de déplacement
Dim_Individu	id_individu, num_individu, sexe, statut matrimonial, profession, permis	Individu
Dim_Mode	id_mode, code_mode, libellé, catégorie_mode	Mode de transport
Dim_Menage	id_menage, num_menage, statut occupation, type logement, équipements	Ménage
Dim_Geographie	id_zone_residence, code_zone, quartier, strate, département	Zone géographique (507 zones)

3.5.4 Hiérarchies

Plusieurs hiérarchies d'agrégation sont exploitables sur ces dimensions : **Dim_Temps** (heure → jour → mois → année), **Dim_Geographie** (zone de résidence → quartier → strate → département), et **Dim_Individu** (individu → ménage). Ces hiérarchies permettent de restituer les indicateurs à différents niveaux de granularité dans le cockpit Knowage, sans nécessiter de retraitement de l'entrepôt.

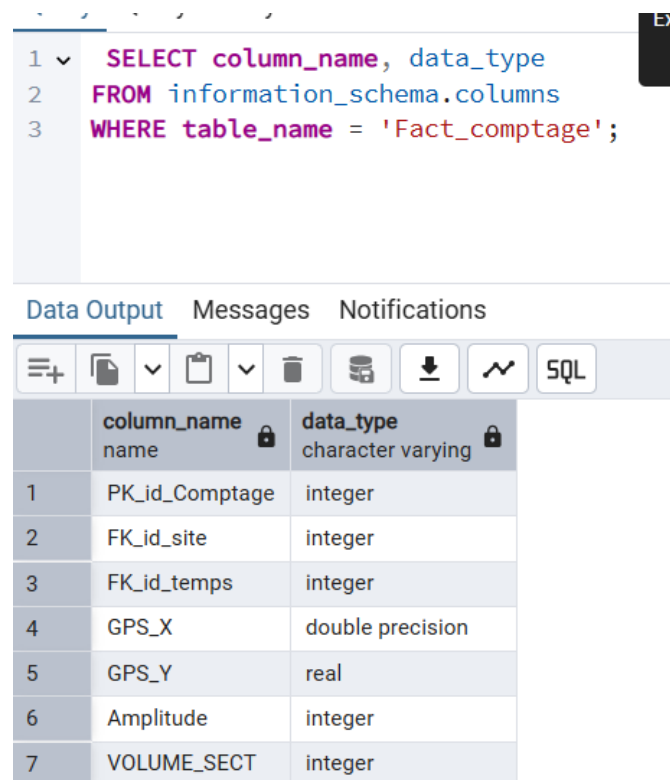
3.5.5 Mesures et indicateurs

Au-delà des mesures directement issues des sources, plusieurs indicateurs calculés ont été intégrés au modèle : le **score d'accessibilité** (0 à 100), composite, intégrant une pénalité liée à la durée de marche jusqu'à l'arrêt le plus proche (jusqu'à -40 points), un bonus de diversité des modes disponibles (jusqu'à +20 points), une pénalité de zone inondable (-25 points, aggravée de -20 points supplémentaires en l'absence de mode disponible en cas de pluie) et une pénalité de tarification abusive en période de pluie (-15 points) ; la **vulnérabilité climatique**, combinant exposition aux inondations et absence de solution de repli ; et l'**impact sur l'éducation**, mesurant le lien entre accessibilité aux transports

et accès aux structures scolaires. Le score d'accessibilité permet de classer chaque zone selon une typologie à quatre niveaux : *Bien desservie* (≥ 75), *Moyenne* (≥ 50), *Sous-desservie* (≥ 25) et *Critique* (< 25).

3.5.6 Implémentation physique

L'entrepôt de données a été déployé dans un schéma dédié (DW) au sein d'une base PostgreSQL. Chaque table de dimension est préfixée `Dim_` et chaque table de faits `Fact_`, conformément aux conventions de nommage usuelles en modélisation décisionnelle. Les clés primaires des dimensions sont des entiers auto-incrémentés (`PK_id_...`), repris en clé étrangère dans les tables de faits (`FK_id_...`). La structure physique de chaque table a été vérifiée via pgAdmin, par interrogation du catalogue système `information_schema.columns`.



The screenshot shows the pgAdmin interface. At the top, an SQL query is entered in the query editor:

```
1 SELECT column_name, data_type
2 FROM information_schema.columns
3 WHERE table_name = 'Fact_comptage';
```

Below the query editor, the 'Data Output' tab is active, displaying the results of the query in a table format. The table has two columns: 'column_name' and 'data_type'. The results are as follows:

	column_name	data_type
1	PK_id_Comptage	integer
2	FK_id_site	integer
3	FK_id_temps	integer
4	GPS_X	double precision
5	GPS_Y	real
6	Amplitude	integer
7	VOLUME_SECT	integer

Figure 3.6: Structure de la table `Fact_Comptage` (pgAdmin)

La dimension géographique (`Dim_Geographie`) illustre la richesse du référentiel mis en place : elle recense 507 zones de résidence, chacune associée à une strate et un quartier de domicile. Une table `fact_access` a par ailleurs été ajoutée au modèle initial afin de porter spécifiquement les mesures liées à l'accessibilité (score d'accessibilité, durée de marche,

nombre de modes disponibles, indicateur de zone inondable), en lien avec les dimensions Dim_Difficulte, Dim_Service et Dim_Transport.

Strate	Quartier_du_domicil	pk_id_zone_residence
MALIKA/YEUMBEUL NORD	262	1
PLATEAU	58	2
MEDINA/GOUNASS	119	3
MALIKA/YEUMBEUL NORD	260	4
CAMBERENE	5	5
DJIDAH/THIAROEY/KAQ/YEUMB SUD	230	6
GUINAW RAIL N - SIPIKINE OUEST	256	7
SAM NOTAIRES	122	8
OUAKAM	44	9
DALIFORD	208	10
FANN/POINT E/AMITIE	14	11
RUFISQUE EST	328	12
JAXAAY PARCELLE NIAKOUL RAP TIVAOUANE PEULH-NIAGHA	311	13
MEDINA	35	14
MEDINA/GOUNASS	116	15
GRAND YOFF	7	16
PATTE D'OIE	54	17
HANN/BEL AIR	63	18

Figure 3.7: Extrait de la table Dim_Geographie (507 zones)

L'outil pgAdmin a été utilisé tout au long du projet pour vérifier la structure des tables après chaque chargement Talend, contrôler les volumes chargés, détecter les valeurs manquantes ou aberrantes, et exécuter des requêtes SQL de contrôle qualité. Ce contrôle a notamment permis d'identifier l'absence de données fiables pour le taux de motorisation des ménages et la mobilité des seniors, indicateurs qui n'ont donc pu être que partiellement renseignés.

3.6 Modèle décisionnel final

Le modèle décisionnel final combine donc : un schéma en constellation à 3 faits / 7 dimensions, des indicateurs calculés (score d'accessibilité, vulnérabilité climatique, impact sur l'éducation) dérivés au chargement, et un référentiel géographique fin (507 zones). Ce modèle constitue le socle physique unique sur lequel s'appuient à la fois le cockpit Knowage BI (chapitre 4) et, indirectement, les analyses préparatoires aux modèles de Machine Learning (chapitre 5), qui s'appuient pour leur part directement sur les sources nettoyées en amont de l'entrepôt.

3.7 Conclusion

Ce chapitre a détaillé la conception décisionnelle de la solution : les sources de données, l'architecture et la mise en œuvre du processus ETL avec Talend, et la conception complète

de l'entrepôt de données PostgreSQL. Ce socle décisionnel alimente directement le cockpit Knowage et l'ensemble des restitutions présentées au chapitre 6. Le chapitre suivant détaille le développement de la plateforme applicative qui expose cette chaîne décisionnelle aux utilisateurs finaux.

Chapter 4

Développement de la plateforme décisionnelle

4.1 Introduction

Conformément à la troisième étape de la méthodologie GIMSI (mise en œuvre), ce chapitre détaille le développement de la plateforme applicative qui expose la chaîne décisionnelle conçue au chapitre précédent : son architecture, le backend FastAPI, le frontend React, l'intégration du cockpit Knowage, et les mesures de sécurité mises en place.

4.2 Architecture de l'application

L'application repose sur une architecture client-serveur classique en trois couches : un **backend** développé avec FastAPI (Python), exposant une dix-huitaine de points d'accès REST avec validation des entrées par schémas Pydantic et documentation interactive automatique (Swagger) ; un **frontend** développé avec React 18, organisé autour d'un routeur central (**App.js**) et d'un contexte de thème (**ThemeProvider**) gérant le mode clair/sombre ; et une couche de **contrôle d'accès** (RBAC) distinguant les rôles *citoyen*, *chef d'exploitation* et *directeur de planification*, chacun disposant d'un périmètre fonctionnel différent. Les deux couches communiquent exclusivement par requêtes HTTP au format JSON, le frontend interrogeant le backend via l'API native **fetch** (aucune bibliothèque tierce de type **axios** n'est utilisée).

4.3 Backend FastAPI

4.3.1 Organisation du code

Le backend est centré sur un unique fichier **main.py** d'environ 820 lignes, structuré de façon linéaire : imports et configuration CORS, chargement de l'ensemble des modèles **.pkl** et fichiers **.json** au démarrage du serveur (dans un bloc **try/except** qui interrompt le démarrage si un fichier de modèle est manquant), dictionnaires métier statiques (quartiers, segments, zones GPS), schémas Pydantic, fonctions utilitaires de construction de features, puis l'ensemble des endpoints. Ce choix de chargement unique au démarrage,

plutôt qu'un chargement à chaque requête, garantit des temps de réponse rapides pour les endpoints faisant appel aux modèles de Machine Learning.

4.3.2 API REST

Le tableau 4.1 récapitule les principaux points d'accès exposés par l'API.

Table 4.1: Principaux endpoints de l'API FastAPI

Endpoint	Fonction
POST /auth/login	Authentification et émission d'un jeton JWT
POST /recommander	Recommandation de mode de transport (citoyen)
GET /zones-risque	Liste des zones à risque d'inaccessibilité, avec coordonnées GPS
GET /zones-risque/resume	Statistiques agrégées et top 5 des zones à risque
GET /api/anomalies/summary	Indicateurs de synthèse sur les anomalies de trafic
GET /api/anomalies/sites	Liste des sites de comptage avec score d'anomalie
GET /api/segmentation/profils	Profils enrichis des cinq segments d'utilisateurs (K-Means)
POST /predict-inaccessibility	Prédiction du score de risque pour un profil donné
GET /api/ml/metrics	Métriques de performance des modèles (accuracy, AUC, F1)
GET /api/ml/features-importance	Importance des variables du modèle de prédiction
GET /api/satisfaction	Indicateurs de satisfaction et d'écoute utilisateurs
GET /quartiers, /segments, /modes	Référentiels statiques utilisés par le frontend

4.3.3 Gestion des utilisateurs et rôles

Les comptes décideurs (chef d'exploitation, directeur de planification) sont gérés côté serveur dans un référentiel `DECIDEURS_DB`, chaque compte étant associé à un mot de passe (haché) et à un rôle. Le rôle conditionne la liste des routes accessibles côté frontend (`allowedRoutes`), contrôlée à la fois lors de la connexion et à chaque navigation protégée.

4.3.4 Authentification

L'authentification s'appuie sur un jeton JWT (JSON Web Token) signé côté serveur (algorithme HS256). L'endpoint `POST /auth/login` vérifie les identifiants transmis contre `DECIDEURS_DB` et renvoie un jeton signé en cas de succès. Ce jeton est ensuite transmis par le frontend dans l'en-tête `Authorization: Bearer <token>` de chaque

requête vers une route protégée, et vérifié côté serveur par une dépendance FastAPI (`get_current_decideur`) injectée sur les endpoints concernés.

4.3.5 Gestion des rôles

Sur l'ensemble des endpoints exposés, treize sont protégés par cette dépendance d'authentification (zones à risque, anomalies, segmentation, métriques de Machine Learning, satisfaction...), tandis que cinq endpoints restent volontairement publics car destinés au citoyen (`/`, `/health`, `/quartiers`, `/recommander`, `/api/feedback`). Une granularité plus fine, distinguant le chef d'exploitation du directeur de planification (par exemple pour l'accès à la page d'audit des modèles), est gérée côté frontend par la liste `allowedRoutes` associée à chaque rôle.

4.3.6 Export PDF

La génération de rapports PDF est réalisée côté client, via la bibliothèque `jsPDF` importée dynamiquement dans le composant `ExportPDF.jsx`. Ce composant permet aux décideurs d'exporter un rapport de synthèse directement depuis les pages *Zones à risque* et *Anomalies de trafic*, sans solliciter le backend.

4.3.7 Communication avec PostgreSQL

Le backend applicatif ne se connecte pas directement à l'entrepôt de données PostgreSQL pour ses fonctionnalités de Machine Learning : ce choix, justifié au chapitre 5, repose sur le fait que les modèles sont entraînés et exploités directement à partir des sources nettoyées. La communication avec PostgreSQL est en revanche centrale pour le cockpit Knowage, qui interroge directement le schéma DW pour produire ses visualisations.

4.3.8 Communication avec les modèles de Machine Learning

Les modèles sérialisés au format `.pkl` via `joblib` (K-Means, Random Forest, modèle d'inaccessibilité par Gradient Boosting) sont chargés en mémoire au démarrage du serveur FastAPI. Les endpoints `POST /recommander` et `POST /predict-inaccessibility` orchestrent ensuite, à chaque requête, la construction des features à partir du profil transmis par l'utilisateur, puis l'appel aux modèles déjà chargés, avant de formater la réponse au format JSON attendu par le frontend.

4.4 Frontend React

4.4.1 Organisation des composants

Le frontend est une application React 18.3.1 générée avec Create React App, structurée en trois grands dossiers : **components/** pour les éléments réutilisables (**ProtectedRoute**, **ThemeToggle**, **LogoTransportDakar**, **LoadingScreen**, **MapView** en Leaflet vanilla, **ExportPDF**), **pages/** pour les écrans applicatifs, et **data/** pour les données statiques locales (**evolutionData.js**). Les bibliothèques principales mobilisées sont **react-router-dom** (routage), **framer-motion** (animations de transition), **lucide-react** (icônes) et **recharts** (graphiques).

4.4.2 Navigation

Le routage est centralisé dans **App.js**. L'arborescence **/decideurs/*** est entièrement enveloppée par le composant **ProtectedRoute**, qui vérifie l'authentification avant de rendre la suite de l'arborescence via **Outlet**. Le contrôle d'accès fin par rôle (par exemple, restreindre la page **MLInsights** au seul directeur de planification) est ensuite géré au niveau de l'interface, dans **DecideursLayout.jsx** et **VueGenerale.jsx**, qui filtrent les éléments de menu et les routes accessibles selon la liste **allowedRoutes** associée à la session de l'utilisateur connecté.

4.4.3 Pages

Le tableau 4.2 récapitule les principales pages de l'application.

Table 4.2: Pages principales du frontend

Page	Fonction	Accès
WelcomePage / FormPage	Accueil et recommandation de mode de transport	Citoyen
AvisCitoyenPage	Cellule d'écoute – dépôt d'avis citoyen	Citoyen
LoginDecideurs	Authentification décideurs	Public
VueGenerale	Tableau de bord de synthèse	Décideurs
ZonesRisquePage	Zones à risque d'inaccessibilité, export PDF	Décideurs
AnomalyDashboard	Détection d'anomalies de trafic	Décideurs
SimulateurRisque	Simulation interactive du score de risque	Décideurs
SegmentationPage	Segments d'utilisateurs (K-Means)	Décideurs
MLInsights	Transparence et audit des modèles de ML	Directeur de planification
EvolutionTemporelle	Analyse prospective 2010–2023	Décideurs
CockpitDakar	Intégration du cockpit Knowage (iframe)	Décideurs

4.4.4 Dashboards et formulaires

Les tableaux de bord décideurs combinent des composants **recharts** (histogrammes, courbes, radars) avec des cartes interactives **MapView** (Leaflet) pour la visualisation géographique des zones à risque et des sites d'anomalies. Le formulaire citoyen (**FormPage**) applique un contrôle de saisie contextualisé : certains champs sont masqués ou adaptés en fonction de l'âge déclaré, afin de garantir la cohérence des données transmises à l'API de recommandation.

4.4.5 Intégration de l'API

Chaque page définit sa propre constante d'URL de base et appelle l'API directement via **fetch** dans un **useEffect**, sans couche d'abstraction centralisée de type **services/**. Ce choix, simple et adapté au périmètre d'un projet de fin d'études, est documenté comme une piste d'amélioration pour une éventuelle mise en production (voir chapitre 6).

4.5 Intégration de Knowage

4.5.1 Datasets

Le cockpit Knowage BI s'appuie sur des *datasets* interrogeant directement le schéma **DW** de l'entrepôt PostgreSQL, un dataset étant généralement associé à une table de faits ou à une agrégation de celle-ci avec ses dimensions.

4.5.2 Cockpits et widgets

Le cockpit est organisé en plusieurs *sheets* thématiques : *Accueil* (vue d'ensemble), *Trafic*, *Déplacements*, *Démographie*, *Accessibilité* et *IA* (restitution synthétique des résultats de Machine Learning). Chaque sheet combine plusieurs widgets (graphiques en barres, courbes, indicateurs numériques) avec une navigation commune et un mode clair/sombre intégral en CSS.

4.5.3 KPI restitués

Les indicateurs restitués couvrent les volumes de trafic (comptages horaires, heures de pointe), les déplacements (durée, coût, motif), la démographie (sexe, statut matrimonial, niveau d'instruction, profession) et l'accessibilité (score composite, zones à risque par typologie).

4.5.4 Intégration dans React

Le cockpit est intégré à l'application React via le composant `CockpitDakar.jsx`, qui l'affiche dans une `iframe` en *same-origin*, grâce à un proxy de développement (`setupProxy.js`) redirigeant les requêtes vers le serveur Knowage local. Cette intégration permet aux décideurs de consulter le cockpit BI sans quitter l'application web principale, dans un environnement visuel cohérent avec le reste de l'application.

4.6 Sécurité de l'application

La sécurité de la plateforme repose sur trois piliers : l'**authentification** par jeton JWT signé (HS256), avec vérification systématique côté serveur pour les routes protégées ; le **contrôle d'accès basé sur les rôles** (RBAC), distinguant citoyen, chef d'exploitation et directeur de planification, appliqué à la fois côté backend (dépendance `get_current_decideur`) et côté frontend (`allowedRoutes`, `ProtectedRoute`) ; et la **validation systématique des entrées** côté backend par les schémas Pydantic, qui rejettent automatiquement toute requête mal formée avant qu'elle n'atteigne la logique métier ou les modèles de Machine Learning.

4.7 Conclusion

Ce chapitre a détaillé le développement de la plateforme applicative : son architecture, le backend FastAPI et ses dix-huit endpoints, le frontend React et son organisation en pages publiques et protégées, l'intégration du cockpit Knowage, et les mécanismes de sécurité mis en place. Le chapitre suivant présente l'intégration des modèles de Machine Learning au sein de cette plateforme, qui constitue l'apport prédictif de la solution.

Chapter 5

Intégration du Machine Learning

5.1 Motivation

Au-delà du reporting descriptif assuré par l'entrepôt de données et le cockpit Knowage, le projet intègre quatre familles de modèles de Machine Learning afin de répondre à des besoins analytiques plus avancés : segmenter les usagers selon leur profil de mobilité, recommander un mode de transport personnalisé, détecter les anomalies de trafic et prédire les zones à risque d'inaccessibilité. Cette dimension prédictive constitue l'apport distinctif du projet par rapport à une solution de Business Intelligence classique, en permettant aux décideurs du CETUD d'anticiper plutôt que de simplement constater.

5.2 Pourquoi un entraînement direct sur les sources

Contrairement au cockpit Knowage, qui interroge l'entrepôt de données PostgreSQL, les modèles de Machine Learning de ce projet sont entraînés et exploités directement à partir des fichiers sources nettoyés (extraits EMD et comptages de trafic), sans passer par l'entrepôt de données. Ce choix est motivé par trois raisons :

- le modèle décisionnel de l'entrepôt a été conçu pour répondre à des besoins de reporting agrégé (KPI, tableaux de bord), et certaines variables individuelles nécessaires aux modèles (par exemple l'ensemble des réponses détaillées de l'EMD utilisées en feature engineering) n'y sont pas toutes représentées à la granularité requise ;
- l'entraînement de modèles de Machine Learning nécessite des itérations rapides sur les données (sélection de variables, comparaison d'algorithmes), un cycle plus proche du travail exploratoire en Python/Colab que des cycles de chargement par lots de l'ETL ;
- découpler les deux chaînes (décisionnelle classique vers le DW, prédictive vers les sources nettoyées) permet de faire évoluer les modèles de Machine Learning indépendamment du calendrier de mise à jour de l'entrepôt de données, sans risque de régression sur le reporting existant.

Ce choix architectural ne remet pas en cause la cohérence de la solution : les sources utilisées par les modèles sont les mêmes données nettoyées qui alimentent par ailleurs

l'ETL Talend, garantissant que les deux chaînes restent fondées sur un référentiel de données identique.

5.3 Pipeline de Machine Learning

Les quatre modèles partagent un pipeline méthodologique commun, décliné selon les spécificités de chaque tâche.

5.3.1 Prétraitement

Les valeurs manquantes sont traitées par imputation : médiane pour les variables numériques (robuste aux valeurs extrêmes), valeur la plus fréquente pour les variables catégorielles. Les variables catégorielles sont encodées, et les variables numériques normalisées (`StandardScaler`) lorsque l'algorithme utilisé est sensible à l'échelle des variables (K-Means notamment).

5.3.2 Feature engineering

Selon le modèle, des variables dérivées sont construites : agrégation de plusieurs indicateurs binaires en un score composite (modèle d'inaccessibilité), construction de statistiques par mode et par segment (durée, coût), ou encore calcul de scores de distance et de densité pour la détection d'anomalies spatiales.

5.3.3 Sélection des variables

La sélection des variables s'appuie sur une combinaison d'expertise métier (variables identifiées comme pertinentes lors de l'analyse des besoins, chapitre 1) et d'analyse statistique (importance des variables calculée a posteriori sur les modèles arborescents, permettant de valider ou d'écarter certaines variables candidates).

5.3.4 Entraînement

Chaque modèle est entraîné avec une validation croisée stratifiée à 5 plis (`StratifiedKFold`) lorsque la tâche est supervisée, garantissant une estimation robuste des performances et limitant le risque de surapprentissage.

5.3.5 Évaluation

Les modèles non supervisés sont évalués par des métriques internes (coefficient de silhouette, méthode du coude). Les modèles supervisés sont évalués par exactitude, F1-score, précision, rappel et aire sous la courbe ROC (ROC AUC), avec une attention particulière à l'équilibre précision/rappel sur les classes minoritaires.

5.3.6 Sauvegarde

Chaque modèle entraîné, ainsi que les objets de prétraitement associés (**scaler**, **encoder**) et les métadonnées (liste des variables, métriques de performance), sont sérialisés au format `.pkl` via `joblib`, afin d'être chargés tels quels par le backend FastAPI sans nouvel entraînement.

5.4 Présentation des modèles

5.4.1 Segmentation des usagers (K-Means)

Objectif. Regrouper les individus enquêtés en profils de mobilité homogènes, afin de permettre des analyses et des recommandations différenciées par segment.

Variables. Variables sociodémographiques et de mobilité de l'enquête individus : âge, sexe, statut d'activité, niveau d'instruction, possession du permis, fréquence d'utilisation des transports en commun, nombre de déplacements, nombre de véhicules, revenu.

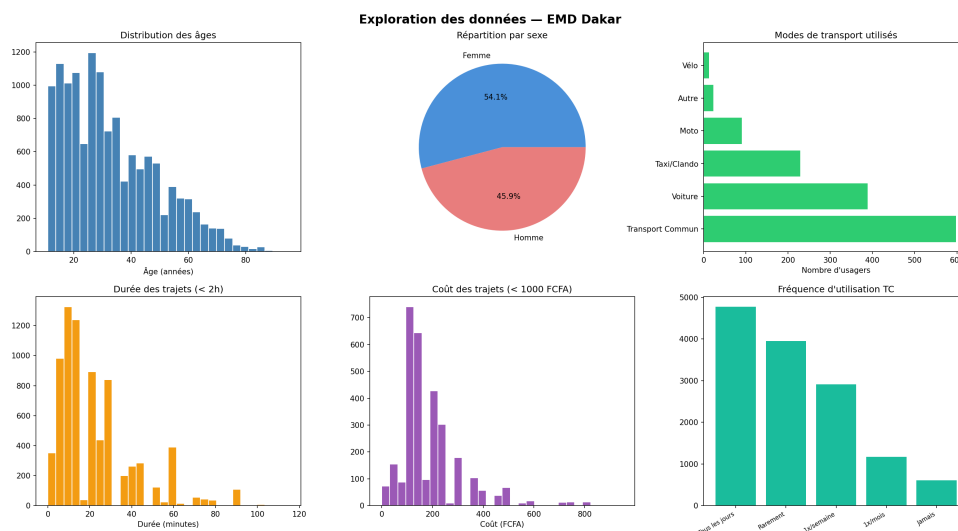


Figure 5.1: Exploration des données individus avant segmentation

Algorithme. `KMeans` (scikit-learn), avec normalisation préalable des variables (`StandardScaler`) afin d'éviter qu'une variable à forte variance (le revenu, notamment) ne domine artificiellement le calcul de distance euclidienne. Le nombre optimal de clusters a été déterminé en combinant la méthode du coude (inertie intra-cluster en fonction de k) et le coefficient de silhouette, conduisant à retenir $k = 5$.

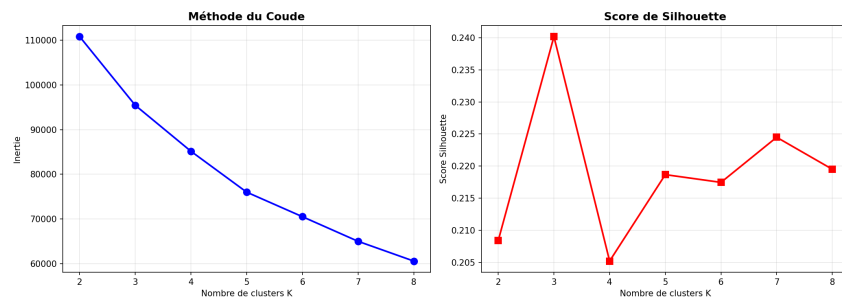


Figure 5.2: Détermination du nombre optimal de clusters (méthode du coude + silhouette)

Résultats. Coefficient de silhouette $\approx 0,31$ pour $k = 5$. Cinq segments ont été identifiés et caractérisés (tableau 5.1) : Actifs TC réguliers, Actifs motorisés, Étudiants mobilité douce, Piétons de proximité, Travailleurs informels.

Table 5.1: Segments d'utilisateurs identifiés par K-Means

Segment	Profil type	Mode dominant
Actifs TC réguliers	~35 ans, actif, sans permis, usage quotidien des TC	Transport commun
Actifs motorisés	~40 ans, actif, permis, revenu élevé	Voiture / Taxi
Étudiants mobilité douce	~20 ans, étudiant, budget faible	Marche / TC
Piétons de proximité	~45 ans, peu mobile, quartier dense	Marche
Travailleurs informels	~30 ans, actif informel, mobilité mixte	TC / Moto

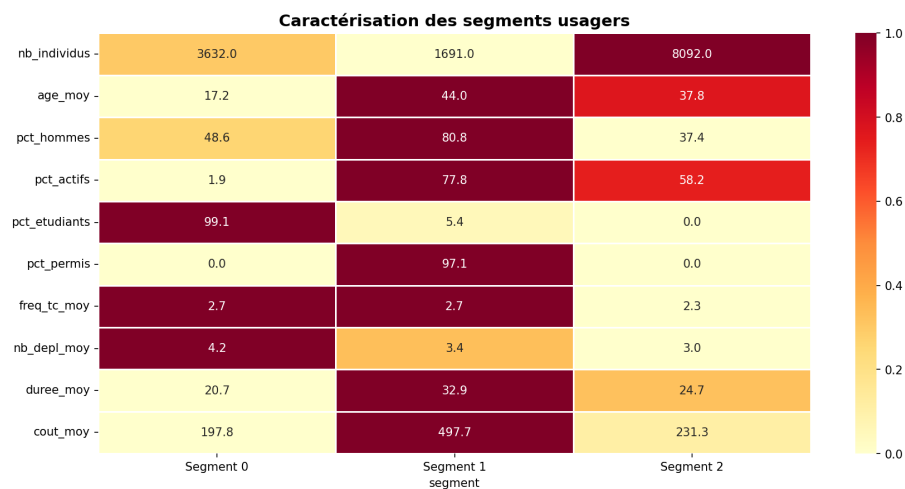


Figure 5.3: Caractérisation des segments par variable (heatmap)

Intégration. Le modèle est exposé via l'endpoint GET `/api/segmentation/profils` et restitué dans la page `SegmentationPage` de l'espace décideurs.

5.4.2 Recommandation de mode de transport (Random Forest)

Objectif. Recommander, pour un profil d'utilisateur donné, le ou les modes de transport les plus adaptés parmi l'ensemble des modes disponibles (transport en commun, voiture, taxi/clando, marche, moto).

Variables. Profil socio-démographique (âge, sexe, statut, niveau d'instruction, permis), habitudes de mobilité (fréquence d'usage des TC, nombre de déplacements, nombre de véhicules), et revenu.

Algorithme. `RandomForestClassifier` (scikit-learn), 200 arbres (`n_estimators=200`), profondeur maximale 10 (`max_depth=10`) pour limiter le surapprentissage, pondération des classes (`class_weight='balanced'`) pour compenser le déséquilibre entre modes, validé par validation croisée stratifiée à 5 plis.

Résultats. Exactitude $\approx 90,6\%$, F1-macro $\approx 0,88$.

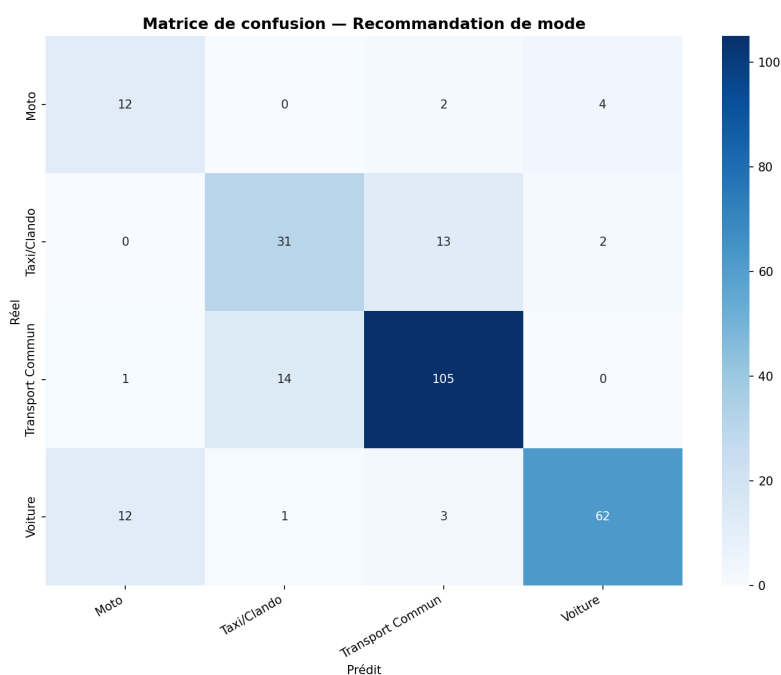


Figure 5.4: Matrice de confusion du modèle de recommandation de mode

Intégration. En production, le modèle retourne, via l'endpoint POST `/recommander`, non pas une seule prédiction mais le *top 3* des modes les plus probables avec leurs probabilités, enrichi par des statistiques de durée et de coût (quartiles Q1/médiane/Q3) cal-

culées par mode et par segment, ce qui permet de fournir au citoyen une recommandation nuancée plutôt qu'une réponse binaire.

5.4.3 Détection d'anomalies de trafic

Objectif. Détecter, parmi les comptages de trafic, des comportements anormaux : pics de volume inattendus, schémas horaires incohérents, sites présentant un comportement différent de leurs voisins géographiques.

Variables. Volumes de comptage par site et par heure, position géographique (GPS) des sites de comptage.

Algorithme. Trois algorithmes complémentaires sont appliqués en parallèle puis combinés par un vote pondéré (tableau 5.2), une observation étant déclarée anormale lorsque le score combiné dépasse le seuil de 0,6. Un clustering spatial DBSCAN est ensuite appliqué sur les coordonnées GPS des sites détectés comme anormaux (rayon $\text{eps} \approx 500$ mètres, minimum 3 sites pour former un groupe), afin de distinguer les *hotspots* de congestion structurelle des incidents ponctuels isolés.

Table 5.2: Algorithmes de détection d'anomalies et pondération du consensus

Algorithme	Poids	Principe
Z-Score	0,25	Détection univariée des valeurs extrêmes par site
Isolation Forest	0,40	Détection multivariée par isolement aléatoire (arbres)
Local Outlier Factor (LOF)	0,35	Détection par densité locale (comparaison aux voisins)

Résultats. Cette approche par consensus réduit significativement les faux positifs par rapport à l'utilisation d'un seul algorithme, chaque méthode ayant des forces complémentaires : le Z-Score est rapide et interprétable mais univarié, l'Isolation Forest est robuste sur des données multivariées sans hypothèse de distribution, et le LOF détecte les anomalies de densité locale invisibles aux deux autres méthodes.



Figure 5.5: Tableau de bord de synthèse des anomalies de trafic détectées

Intégration. Les résultats sont exposés via les endpoints GET `/api/anomalies/summary` et GET `/api/anomalies/sites`, et restitués dans la page `AnomalyDashboard` avec drill-down par site de comptage.

5.4.4 Prédiction du risque d'inaccessibilité

Objectif. Prédire, pour une zone ou un ménage donné, le risque d'inaccessibilité aux transports en commun, afin d'orienter les décisions d'investissement en infrastructure.

Variables. Sept indicateurs binaires dérivés des variables de l'EMD : distance à pied jusqu'à l'arrêt de transport en commun supérieure à 10 minutes, fréquence d'inondation élevée du quartier, enclavement déclaré, absence de routes carrossables, manque de transport en commun déclaré, inaccessibilité du transport en commun en cas de pluie, et durée d'accès au centre de santé supérieure à 30 minutes. Ces sept indicateurs sont combinés en un score composite, dont le seuil de classification (ménage à risque ou non) est fixé au

65^e percentile.

Algorithme. Le pipeline complet comprend l'imputation des valeurs manquantes, l'encodage des variables catégorielles, la normalisation des variables numériques, puis un rééchantillonnage par sur-échantillonnage synthétique (**SMOTE**) afin de compenser le déséquilibre entre ménages à risque (minoritaires) et ménages non à risque. Quatre algorithmes ont été comparés par validation croisée stratifiée : régression logistique, Random Forest, Gradient Boosting et XGBoost. Le **Gradient Boosting** a été retenu comme modèle final, offrant les meilleures performances sur ce jeu de données déséquilibré.

Résultats. Le tableau 5.3 synthétise les performances obtenues.

Table 5.3: Performances du modèle de prédiction d'inaccessibilité (Gradient Boosting)

Métrique	Valeur
Exactitude (Accuracy)	90,6 %
ROC AUC	0,921
F1-score	0,883
Précision	$\approx 89\%$
Rappel	$\approx 88\%$

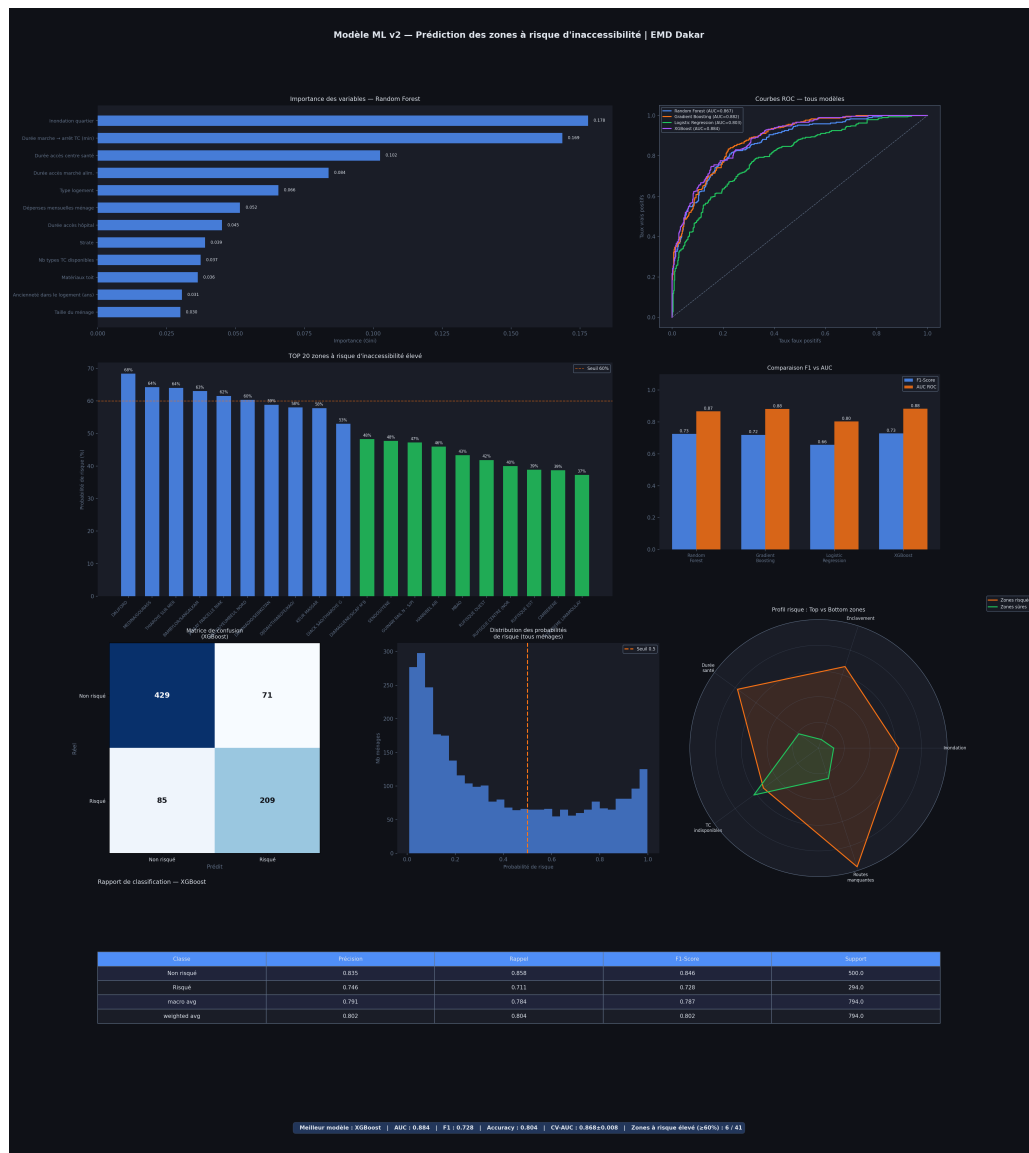


Figure 5.6: Cartographie des zones classées par niveau de risque d'inaccessibilité

L'analyse de l'importance des variables (figure 5.7) confirme que la distance à pied jusqu'à l'arrêt de transport en commun et la durée d'accès aux soins de santé sont les déterminants principaux du risque d'inaccessibilité, en cohérence avec la littérature sur la mobilité urbaine en Afrique subsaharienne.

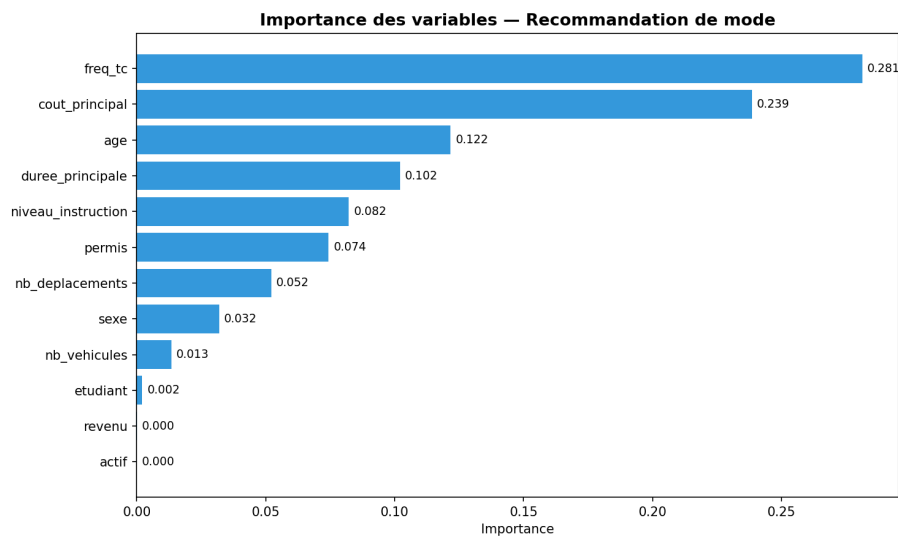


Figure 5.7: Importance des variables du modèle de prédiction d'inaccessibilité

Une analyse complémentaire par valeurs de Shapley (SHAP) a été menée afin de renforcer l'explicabilité du modèle auprès des décideurs (figure 5.8), en détaillant, pour chaque prédiction individuelle, la contribution positive ou négative de chaque variable au score de risque.

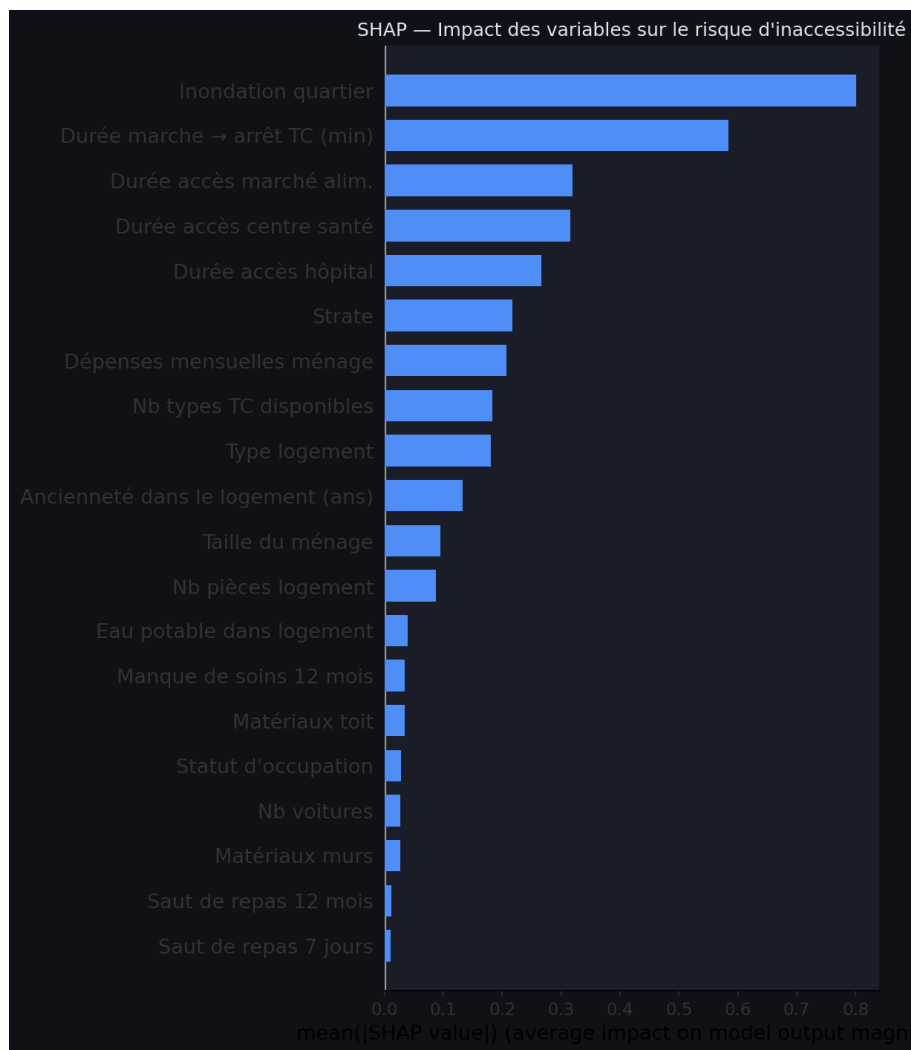


Figure 5.8: Analyse d'explicabilité SHAP du modèle de prédiction d'inaccessibilité

Intégration. Le modèle est exposé via l'endpoint `POST /predict-inaccessibility`, et restitué à la fois dans la page `ZonesRisquePage` (cartographie des zones) et dans la page `SimulateurRisque`, qui permet à un décideur de faire varier les sept indicateurs pour simuler le score de risque d'une zone hypothétique.

5.5 Intégration dans FastAPI

Les quatre modèles, ainsi que leurs objets de prétraitement et métadonnées, sont chargés en mémoire une seule fois au démarrage du serveur FastAPI, dans un bloc de chargement global qui interrompt volontairement le démarrage si un fichier de modèle venait à manquer (afin d'éviter de servir des prédictions incohérentes). Les endpoints orchestrent ensuite, à chaque requête, la construction des features à partir des données transmises, l'appel au modèle déjà chargé, puis le formatage de la réponse au format JSON attendu par le frontend.

5.6 Exploitation des prédictions

Du côté citoyen, les prédictions du modèle de recommandation alimentent directement le parcours de planification de trajet (`FormPage`). Du côté décideurs, les prédictions des trois autres modèles sont exploitées de façon complémentaire : la segmentation guide la priorisation des actions de communication et d'adaptation de l'offre par profil d'utilisateur ; la détection d'anomalies alerte sur les dysfonctionnements de trafic nécessitant une intervention opérationnelle ; et la prédiction du risque d'inaccessibilité oriente les décisions d'investissement en infrastructure, avec un outil de simulation permettant d'évaluer l'impact attendu d'un projet avant sa réalisation.

5.7 Valeur ajoutée de l'intelligence artificielle

L'intégration de ces quatre modèles transforme la solution d'un outil de reporting descriptif en un outil d'aide à la décision véritablement prédictif et prescriptif : au lieu de seulement constater des volumes de trafic ou des taux d'accessibilité passés, le système est désormais capable d'anticiper les comportements (segmentation, recommandation) et les risques (anomalies, inaccessibilité), avec un niveau de transparence (importance des variables, explicabilité SHAP) qui permet aux décideurs du CETUD d'auditer et de faire confiance aux résultats produits.

5.8 Conclusion

Ce chapitre a présenté l'intégration du Machine Learning au sein de la solution : la justification du choix d'un entraînement direct sur les sources, le pipeline méthodologique commun aux quatre modèles, leur présentation détaillée (segmentation, recommandation, détection d'anomalies, prédiction d'inaccessibilité), leur intégration technique dans le backend FastAPI, et la valeur ajoutée qu'ils apportent aux décideurs du CETUD. Le chapitre suivant présente les résultats globaux obtenus et leur évaluation.

Chapter 6

Résultats et évaluation

6.1 Introduction

Conformément à la quatrième étape de la méthodologie GIMSI (amélioration permanente), ce chapitre présente une évaluation transversale des résultats obtenus : les tableaux de bord produits, l'analyse des indicateurs clés, les résultats des modèles de Machine Learning, une comparaison entre l'apport de la Business Intelligence classique et celui du Machine Learning, l'apport global de la solution, ainsi que ses limites et perspectives d'amélioration.

6.2 Présentation des dashboards

Le cockpit Knowage BI restitue l'ensemble des indicateurs descriptifs de l'entrepôt de données à travers cinq sheets thématiques : *Accueil* (vue de synthèse), *Trafic*, *Déplacements*, *Démographie* et *Accessibilité*, complétées par un sheet *IA* restituant de façon synthétique les résultats des modèles de Machine Learning. Côté application web, les décideurs disposent en complément de tableaux de bord dédiés (*VueGenerale*, *ZonesRisquePage*, *AnomalyDashboard*, *SegmentationPage*) permettant une navigation et un drill-down plus fins que dans le cockpit BI seul.

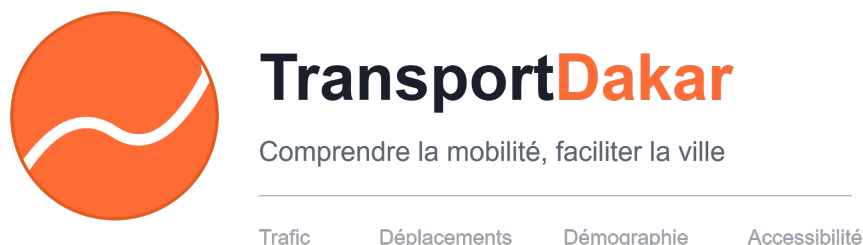


Figure 6.1: Identité visuelle du cockpit décisionnel TransportDakar

6.3 Analyse des KPIs

L'entrepôt de données alimente un ensemble cohérent d'indicateurs clés, synthétisés au tableau 6.1.

Table 6.1: Synthèse des indicateurs clés produits par la solution

Indicateur	Valeur
Individus enquêtés (EMD)	37 248
Ménages enquêtés (EMD)	4 521
Sites de comptage de trafic	49
Observations de trafic chargées	775 480
Zones géographiques référencées	507

L'analyse de ces indicateurs met en évidence des disparités marquées de mobilité et d'accessibilité selon les zones : certains quartiers périphériques cumulent un score d'accessibilité faible, une forte exposition aux inondations et une dépendance quasi exclusive à la marche, tandis que les zones centrales bénéficient d'une offre de transport diversifiée et d'un score d'accessibilité élevé.

6.4 Résultats des modèles ML

Le tableau 6.2 récapitule les performances des quatre modèles de Machine Learning présentés au chapitre 5.

Table 6.2: Synthèse des performances des modèles de Machine Learning

Modèle	Type	Objectif	Performance
K-Means ($k = 5$)	Non supervisé	Segmentation des usagers	Silhouette $\approx 0,31$
Random Forest	Supervisé	Recommandation de mode	Accuracy $\approx 90,6\%$, F1-macro $\approx 0,88$
Z-Score + Isolation Forest + LOF + DBSCAN	Détection d'anomalies	Anomalies de trafic	Consensus pondéré, seuil 0,6
Gradient Boosting	Supervisé	Prédiction d'inaccessibilité	ROC AUC = 0,921, F1 = 0,883

Ces performances, toutes supérieures à 88% en F1-score ou en exactitude pour les modèles supervisés, témoignent de la pertinence des variables sélectionnées et de la qualité du travail de prétraitement effectué en amont.

6.5 Comparaison BI vs ML

Le tableau 6.3 compare les apports respectifs du volet Business Intelligence classique (ETL, entrepôt, cockpit Knowage) et du volet Machine Learning de la solution.

Table 6.3: Comparaison des apports de la BI classique et du Machine Learning

Critère	Business Intelligence (Knowage)	Machine Learning
Nature de l'analyse	Descriptive (constat, agrégation)	Prédictive et exploratoire (anticipation, segmentation)
Horizon	Passé / présent	Présent / futur probable
Granularité	Agrégée (zone, période)	Individuelle (ménage, site, déplacement)
Explicabilité	Élevée (requêtes SQL directes)	Variable (nécessite des outils dédiés, ex. SHAP)
Mise à jour	Par lot ETL (cycle programmé)	Ré-entraînement périodique sur les sources
Valeur pour le décideur	Suivi, reporting régulier	Aide à la décision, simulation, alerte

Ces deux volets ne sont pas concurrents mais complémentaires : la Business Intelligence répond aux besoins de suivi régulier et de gouvernance des indicateurs, tandis que le Machine Learning apporte une capacité d'anticipation que le reporting descriptif ne peut pas offrir par construction.

6.6 Apport global

La solution développée transforme la gestion de la donnée du CETUD à trois niveaux : elle **centralise** des données auparavant dispersées et exploitées de façon ponctuelle dans un entrepôt de données structuré et documenté ; elle **automatise** la production des indicateurs clés, qui étaient auparavant recalculés manuellement à chaque besoin ; et elle **enrichit** l'analyse par des capacités prédictives (segmentation, détection d'anomalies, prédiction de risque) absentes du dispositif antérieur. L'intégration de l'ensemble dans une application web unique, accessible aussi bien au grand public qu'aux décideurs du CETUD, constitue enfin un apport en termes de diffusion et d'appropriation de la donnée, au-delà du seul cercle des experts techniques.

6.7 Limites

Le système présente certaines limites, assumées au regard du cadre d'un projet de fin d'études :

- l'absence de données de trafic en temps réel, le système s'appuyant sur des comptages historiques ;
- l'utilisation de l'EMD la plus récente disponible publiquement, qui ne reflète pas nécessairement l'évolution la plus actuelle de la mobilité à Dakar ;
- une authentification fonctionnelle mais simplifiée par rapport aux standards de production (absence de rotation de jetons, de gestion fine des sessions multiples) ;
- l'absence de tests automatisés et de pipeline d'intégration continue ;
- une architecture frontend qui duplique la constante d'URL de l'API dans chaque page, en l'absence d'une couche de services centralisée.

6.8 Perspectives

Ces limites ouvrent autant de pistes d'amélioration pour une éventuelle mise en production : intégration de données de trafic en temps réel issues des opérateurs de transport, conteneurisation de la solution (Docker), renforcement de l'authentification, mise en place de tests automatisés et de pipelines d'intégration continue, centralisation des appels API côté frontend, et, à plus long terme, exploration de techniques de Deep Learning (réseaux de neurones récurrents pour la prédiction de trafic, réseaux de neurones sur graphes pour la modélisation du réseau de transport).

6.9 Conclusion

Ce chapitre a permis d'évaluer de façon transversale les résultats du projet : les dashboards produits, l'analyse des KPI, les performances des modèles de Machine Learning, leur complémentarité avec la Business Intelligence classique, ainsi que l'apport global de la solution et ses limites. Ces éléments alimentent directement le bilan dressé dans la conclusion générale.

Conclusion générale

Bilan

Ce projet de fin d'études a porté sur la conception et la mise en œuvre d'une solution décisionnelle complète au service du CETUD, visant à exploiter l'Enquête Ménages-Déplacements et les données de comptage de trafic de Dakar pour éclairer la prise de décision en matière de mobilité urbaine. La démarche suivie s'est appuyée sur la méthodologie GIMSI, qui a structuré le travail en quatre grandes étapes : l'identification des besoins et des acteurs (chapitre 1), la conception de la solution et du modèle décisionnel (chapitres 2 et 3), la mise en œuvre technique de la plateforme et des modèles prédictifs (chapitres 4 et 5), et l'évaluation des résultats obtenus (chapitre 6).

Contributions

Au-delà du volet décisionnel classique (ETL Talend, entrepôt de données en schéma constellation, cockpit Knowage BI), le projet a été enrichi par le développement de quatre modèles de Machine Learning : la segmentation des usagers par K-Means, la recommandation de mode de transport par Random Forest, la détection d'anomalies de trafic par un consensus de trois algorithmes (Z-Score, Isolation Forest, Local Outlier Factor) complété par un clustering spatial DBSCAN, et la prédiction du risque d'inaccessibilité par Gradient Boosting. Ces modèles ont été exposés au sein d'une application web complète, combinant un backend FastAPI sécurisé par authentification JWT et un frontend React, à destination à la fois des citoyens et des décideurs du CETUD (chef d'exploitation et directeur de planification). Sur le plan des compétences acquises, ce projet a permis une mise en pratique transversale couvrant l'ensemble de la chaîne de valeur de la donnée : de la modélisation décisionnelle classique jusqu'aux techniques avancées de Machine Learning, en passant par le développement complet d'une application web sécurisée. Cette double dimension, décisionnelle et prédictive, constitue l'apport principal du projet par rapport à une solution de reporting classique.

Difficultés rencontrées

La conduite de ce projet a confronté à plusieurs difficultés caractéristiques d'un projet décisionnel de cette ampleur : l'hétérogénéité et l'incomplétude de certaines variables

sources de l'EMD (taux de motorisation des ménages et mobilité des seniors notamment, partiellement renseignés faute de données fiables) ; l'arbitrage entre granularité fine des données individuelles et performance des agrégations dans l'entrepôt, qui a motivé le choix du schéma en constellation plutôt qu'un schéma en étoile unique ; le déséquilibre des classes dans la tâche de prédiction d'inaccessibilité, traité par sur-échantillonnage SMOTE ; et la nécessité de concilier deux chaînes de traitement distinctes (ETL vers l'entrepôt pour la restitution BI, entraînement direct sur les sources pour le Machine Learning) sans rompre la cohérence d'ensemble de la solution.

Perspectives d'amélioration

Le système présente des limites assumées au regard du cadre d'un projet de fin d'études : l'absence de données de trafic en temps réel, l'utilisation de l'EMD la plus récente disponible publiquement, une authentification fonctionnelle mais simplifiée, et l'absence de tests automatisés. Ces limites ouvrent autant de pistes d'amélioration pour une mise en production : intégration de données temps réel issues des opérateurs de transport, conteneurisation Docker, renforcement de l'authentification, mise en place de tests automatisés et de pipelines d'intégration continue, et, à plus long terme, exploration de techniques de Deep Learning (réseaux de neurones récurrents pour la prédiction de trafic, réseaux de neurones sur graphes pour la modélisation du réseau de transport).

En définitive, ce projet démontre la faisabilité et l'intérêt d'une approche combinant Business Intelligence traditionnelle et Machine Learning pour répondre aux enjeux de mobilité urbaine d'une métropole comme Dakar, et pose les bases d'un système qui pourra être consolidé et étendu dans le cadre d'une collaboration future avec le CETUD.

Bibliography

- [1] CETUD – Conseil Exécutif des Transports Urbains de Dakar, *Enquête Ménages-Déplacements de Dakar*.
- [2] Fernandez, A., *Les Nouveaux Tableaux de Bord des Managers : la méthode GIMSI*, Éditions Eyrolles.
- [3] Kimball, R., Ross, M., *The Data Warehouse Toolkit : The Definitive Guide to Dimensional Modeling*, Wiley.
- [4] Inmon, W. H., *Building the Data Warehouse*, Wiley.
- [5] Talend, *Talend Open Studio – Documentation officielle*, <https://www.talend.com/>.
- [6] PostgreSQL Global Development Group, *PostgreSQL Documentation*, <https://www.postgresql.org/docs/>.
- [7] Knowage Labs, *Knowage Documentation*, <https://www.knowage-suite.com/>.
- [8] Pedregosa, F. et al., *Scikit-learn : Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, 2011.
- [9] Ramírez, S., *FastAPI Documentation*, <https://fastapi.tiangolo.com/>.
- [10] Meta Open Source, *React – Documentation officielle*, <https://react.dev/>.
- [11] Pydantic, *Pydantic Documentation*, <https://docs.pydantic.dev/>.
- [12] Jones, M., Bradley, J., Sakimura, N., *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force, 2015.
- [13] Breiman, L., *Random Forests*, Machine Learning, vol. 45, no. 1, 2001.
- [14] MacQueen, J., *Some Methods for Classification and Analysis of Multivariate Observations*, Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, 1967.
- [15] Liu, F. T., Ting, K. M., Zhou, Z.-H., *Isolation Forest*, IEEE International Conference on Data Mining, 2008.
- [16] Breunig, M. M., Kriegel, H.-P., Ng, R. T., Sander, J., *LOF : Identifying Density-Based Local Outliers*, ACM SIGMOD, 2000.

- [17] Ester, M., Kriegel, H.-P., Sander, J., Xu, X., *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise*, KDD, 1996.
- [18] Friedman, J. H., *Greedy Function Approximation : A Gradient Boosting Machine*, Annals of Statistics, vol. 29, no. 5, 2001.
- [19] Chawla, N. V. et al., *SMOTE : Synthetic Minority Over-sampling Technique*, Journal of Artificial Intelligence Research, vol. 16, 2002.
- [20] Lundberg, S. M., Lee, S.-I., *A Unified Approach to Interpreting Model Predictions*, Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [21] Schwaber, K., Sutherland, J., *The Scrum Guide*, <https://scrumguides.org/>.

Appendix A

Annexes

A.1 Scripts SQL

Extrait de requête de contrôle qualité exécutée sous pgAdmin pour vérifier la structure d'une table de faits via le catalogue système :

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'DW' AND table_name = 'Fact_Comptage'
ORDER BY ordinal_position;
```

Extrait de requête de contrôle de cohérence référentielle (détection de clés étrangères orphelines) :

```
SELECT f.*
FROM "DW"."Fact_deplacement" f
LEFT JOIN "DW"."Dim_Individu" d ON f."FK_id_individu" = d."PK_id_individu"
WHERE d."PK_id_individu" IS NULL;
```

A.2 Jobs Talend

Le tableau A.1 récapitule l'ensemble des jobs Talend développés pour l'alimentation de l'entrepôt de données.

Table A.1: Jobs Talend développés

Job	Rôle
SA_individu, SA_menage, SA_deplacement	Chargement de la Staging Area
Job_dim_site, Job_dim_geo	Construction des dimensions de référence
Job_dim_individu, Job_dim_menage	Construction des dimensions individu / ménage
Job_deplacement	Construction de Fact_deplacement
Job_comptage	Construction de Fact_Comptage
Job_accessibilite	Construction de fact_access
Job_indivmen	Construction de Fait_IndividuMenage

A.3 Architecture complète

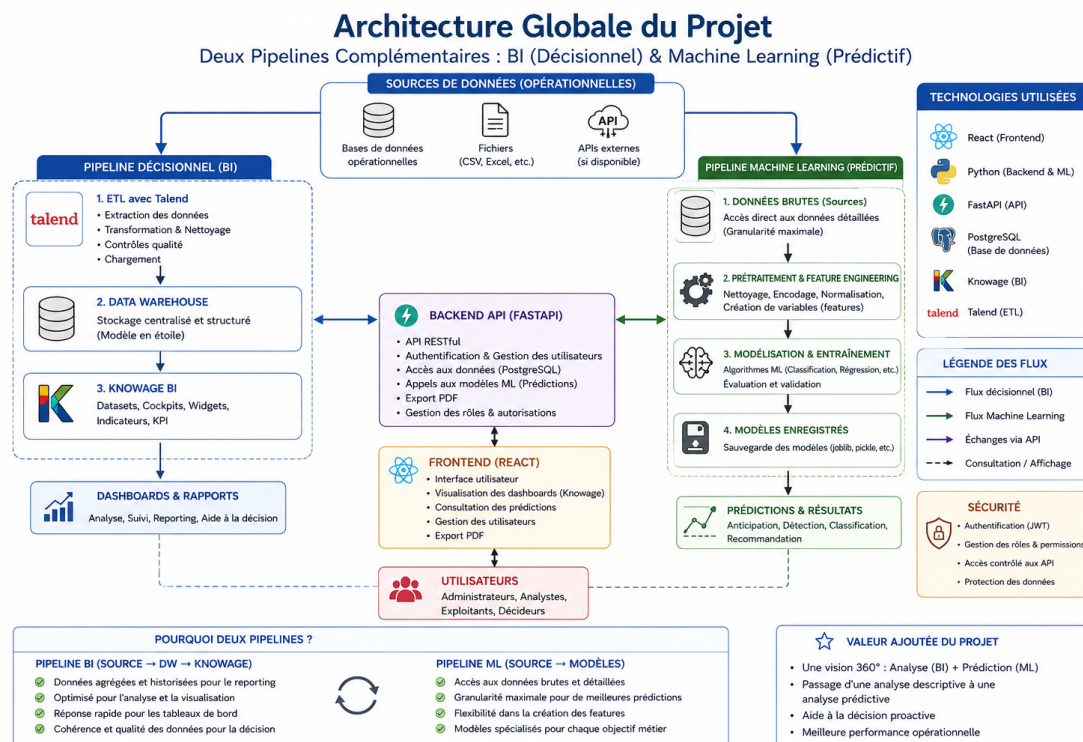


Figure A.1: Architecture technique complète de la solution

A.4 Captures Knowage

Les sheets du cockpit Knowage BI (Accueil, Trafic, Déplacements, Démographie, Accessibilité, IA) sont illustrées par les figures du chapitre 6 ainsi que par les captures du dossier `figures/kpis/` du présent rapport.

A.5 Captures React

Les pages de l'application web (espace citoyen et espace décideurs) sont illustrées par les figures du chapitre 4, présentant respectivement le parcours citoyen (recommandation de mode de transport) et les tableaux de bord décideurs (zones à risque, anomalies, segmentation, simulateur de risque).

A.6 Documentation de l'API

Le tableau A.2 liste l'ensemble des points d'accès REST exposés par le backend FastAPI, avec leur méthode HTTP et leur statut de protection.

Table A.2: Liste complète des endpoints REST exposés par l'API

Méthode	Endpoint	Protégé (JWT)
POST	/auth/login	—
GET	/	Non
GET	/health	Non
GET	/quartiers	Non
POST	/recommander	Non
GET	/segments	Oui
GET	/modes	Oui
GET	/zones-risque	Oui
GET	/zones-risque/resume	Oui
GET	/api/segmentation/profils	Oui
GET	/api/satisfaction	Oui
POST	/api/feedback	Non
GET	/api/feedback/stats	Oui
GET	/api/anomalies/summary	Oui
GET	/api/anomalies/sites	Oui
GET	/api/anomalies/sites/{site_id}/details	Oui
POST	/predict-inaccessibility	Oui
GET	/api/ml/features-importance	Oui
GET	/api/ml/metrics	Oui

La documentation interactive complète (Swagger UI) est générée automatiquement par FastAPI et accessible à l'adresse `/docs` du serveur.

A.7 Documentation des modèles de Machine Learning

Chaque modèle est sérialisé avec ses métadonnées (liste des variables, métriques de performance, date d'entraînement) au format `.pkl` via `joblib` : `kmeans_segmentation.pkl`, `rf_recommandation.pkl`, `inaccessibilite_model.pkl`, ainsi que les fichiers de métadonnées associés (`metadata.json`, `zones_risque.json`). Le détail méthodologique de chaque modèle (objectif, variables, algorithme, résultats, intégration) est présenté au chapitre 5.

A.8 Manuel utilisateur

A.8.1 Espace citoyen

1. Accéder à la page d'accueil de l'application (`WelcomePage`) ;
2. Cliquer sur « Planifier mon trajet » pour accéder au formulaire de recommandation ;

3. Renseigner son profil de mobilité (zone de résidence, âge, statut, disponibilité d'un véhicule, fréquence des déplacements) ;
4. Consulter le *top 3* des modes de transport recommandés, avec durée et coût estimés.

A.8.2 Espace décideurs

1. Accéder à la page de connexion (**LoginDecideurs**) et s'authentifier avec ses identifiants (chef d'exploitation ou directeur de planification) ;
2. Consulter la vue générale (**VueGenerale**) pour un aperçu synthétique des indicateurs clés ;
3. Naviguer vers les modules thématiques selon ses besoins : zones à risque, anomalies de trafic, segmentation des usagers, simulateur de risque, cockpit Knowage ;
4. Pour le directeur de planification uniquement : consulter la page de transparence des modèles (**MLInsights**) pour auditer les performances et l'importance des variables ;
5. Exporter un rapport PDF de synthèse depuis les pages *Zones à risque* ou *Anomalies* si nécessaire.

A.9 Dictionnaire des codes de variables EMD utilisés

Table A.3: Principaux codes de variables EMD exploités

Code	Signification
I2, I4, I9	Caractéristiques géographiques du domicile (quartier, strate, district)
M21	Taille du ménage
M55	Manque de transport en commun déclaré
M66 – M70	Accessibilité de base (distance, durée de marche, modes disponibles)
M68	Exposition aux inondations
M69	Résilience d'accès en cas de pluie
M73, M87	Problèmes majeurs déclarés dans le quartier
M90, M91	Accès aux écoles et daaras

A.10 Glossaire

BI Business Intelligence – informatique décisionnelle.

CETUD Conseil Exécutif des Transports Urbains de Dakar.

DBSCAN Density-Based Spatial Clustering of Applications with Noise – algorithme de clustering spatial.

EMD Enquête Ménages-Déplacements.

ETL Extract-Transform-Load – processus d'extraction, transformation et chargement des données.

GIMSI Méthodologie de conception de tableaux de bord décisionnels.

JWT JSON Web Token – jeton signé utilisé pour l'authentification.

KPI Key Performance Indicator – indicateur clé de performance.

LOF Local Outlier Factor – algorithme de détection d'anomalies par densité locale.

RBAC Role-Based Access Control – contrôle d'accès basé sur les rôles.

SHAP SHapley Additive exPlanations – méthode d'explicabilité des modèles de Machine Learning.

SMOTE Synthetic Minority Over-sampling Technique – technique de rééquilibrage de classes.

A.11 Liste des technologies utilisées

Table A.4: Stack technique du projet

Couche	Technologie
Préparation des données	Python, Pandas, Google Colab
ETL	Talend Open Studio
Entrepôt de données	PostgreSQL
Restitution décisionnelle	Knowage BI
Machine Learning	scikit-learn, imbalanced-learn, SHAP, joblib
Backend applicatif	FastAPI, Pydantic, PyJWT, uvicorn
Frontend applicatif	React.js, Recharts, Leaflet, Framer Motion, jsPDF
Gestion de projet	Scrum